

INTERVIEW PREPARATION PACK

Tailored to your resume, projects, and the target role

Field	Detail
Candidate	Ravi Ranjan Prasad
Current role	AI Backend Engineer — Tata Consultancy Services (TCS) Oct 2021 – Present
Total experience	6 years Wipro Jun 2019 – Jun 2020 TCS Oct 2021 – Present
Domain	Enterprise Conversational AI, GenAI assistants, RAG, Azure GenAI services
Anchor project	ITChat — ExxonMobil Enterprise AI Assistant (FastAPI + Node.js, RAG, ServiceNow, MS Teams)
Base location	Faridabad, Haryana (Delhi NCR) willing to relocate for the right role
Target role	AI Chatbot Developer — Co-Pilot Studio & RAG Specialist (TVS Group)
Core stack	Python FastAPI Node.js/TypeScript NestJS LangChain Azure OpenAI Azure AI Search
Certifications	Claude Certified Developer – Foundations Claude Certified Architect – Foundations (both valid to 2027)
	Microsoft Certified: Azure AI Fundamentals (AI-900) Azure Fundamentals (AZ-900)

READ THIS FIRST — WHAT CHANGED FROM YOUR OLD PACK

This version is rebuilt from your verified profile, not a generic template. Four corrections matter most:

1. You have FOUR certifications (2x Claude, AI-900, AZ-900). The old pack scripted "I have no certifications" — that would now cost you points. Never say it.
2. ITCat started in SEPTEMBER 2024, not 2025. That makes it roughly 25 months of continuous ownership, and it makes your GenAI experience five years rather than four. Do not shorten it.
3. Your Shell stint was one continuous engagement from Oct 2021 to Sep 2024 at Shell India Markets — about three years — not two separate Shell entities. Present it as one engagement.
4. You have a ~15-month freelance period between Wipro and TCS (Jul 2020 – Sep 2021). Do not hide it and do not over-explain it. There is a scripted answer in Batch 1.

Compiled from your resume | Anchor JD: AI Chatbot Developer — Co-Pilot Studio & RAG Specialist, TVS Group

HOW TO USE THIS PACK

This is a working document, not something to read end to end. Use it in the sequence below, and treat the shaded SPOKEN SCRIPT blocks as verbatim lines to memorise and rehearse out loud.

Stage	What to do	Time needed
Stage 1	Read Batch 1 in full. Memorise the 75-second introduction and the 90-second ITChat description word for word.	1 hour
Stage 2	Work through Batch 2 (GenAI + RAG). This is the round you are scored hardest on. Rehearse the RAG pipeline, chunking and hallucination answers aloud.	3–4 hours
Stage 3	Batch 3 — rehearse the multi-tenant architecture with a pen and paper. Then work the Top Twelve Questions with answers, and the Trending 2026 section.	3 hours
Stage 4	Batch 4 — rehearse your three STAR stories until you can deliver each in 90 seconds without notes.	2 hours
Stage 5	Work the Final section: score yourself against the rubric, run a mock with a friend, then use the checklist the night before.	2 hours

The three rules that decide a senior GenAI interview

1. Specificity beats seniority language. "Azure AI Search with hybrid vector + BM25 retrieval and semantic reranking" is worth more than "strong RAG expertise". Name the service, the pattern, the number.
2. Never bluff a tool you have not used. The one gap you have (Co-Pilot Studio) is a two-week ramp, and honesty about it reads as senior. Bluffing it, if caught, ends the interview.
3. Every technical answer ends with a business outcome. Accuracy percentage, deflection rate, latency, ticket reduction, cost per conversation. Engineering without outcomes reads as a hobbyist.

A NOTE ON YOUR DELIVERY

Your resume shows real, verifiable production work across three enterprises and two employers. Most candidates at your level are padding a single project. You do not need to.

Speak in first person, present tense for current work, past tense for shipped work, and never hedge with "I think maybe". If you do not know something, say "I have not used that in production — here is how I would approach it" and then actually approach it.

YOUR VERIFIED PROFILE The single source of truth

Everything in this pack traces back to this table. If a fact is not here, do not claim it. If a fact is here, be ready to defend it in detail — the panel will probe.

Experience timeline — know these durations cold

Period	Employer / Client	Role	Duration	What you must be able to defend
Sep 2024 – Present	TCS — ExxonMobil Services and Technology — ITChat	AI Backend Engineer / Architect	~25 mo	Enterprise chatbot architecture, GenAI IT assistant, RAG pipeline, Genesys Live Chat integration, ServiceNow REST and Catalog, MS Teams, Azure App Service and Functions
Oct 2021 – Sep 2024	TCS — Shell — Shell India Markets	AI Backend Engineer	~3 yrs	Microsoft Bot Framework chatbot modules, MongoDB query optimisation, Redis caching layers, ServiceNow (Ticketing, KB), Azure CLU intent models, Adaptive Cards, Genesys Live Chat escalation
Jul 2020 – Sep 2021	Independent — Freelance video editor and creator	Freelance video editor / Creator	~15 mo	Freelance video editing; built the PsycheWizard motivational account to roughly 600,000 followers. Be ready to explain this period in one clean sentence — see the note below.
Jun 2019 – Jun 2020	Wipro — IAF-Maximo (Asset Management) and Core Integration Stream (IAF-AIC)	Associate Software Engineer 2	~1 yr	IBM Maximo EAM, Java components, SQL/MySQL optimisation, integration validation, enterprise asset tracking backend support, database optimisation at scale

NUMBERS TO GET RIGHT BEFORE YOU WALK IN

ITChat at ExxonMobil has been running since September 2024, so it is roughly twenty-five months of continuous ownership. That is the number that matters most, because long tenure on one production system is what proves you owned it through multiple iterations rather than building a demo.

Sam at Shell ran from October 2021 to September 2024 — about three years on one chatbot, through Bot Framework, Azure CLU intent models, ServiceNow and Genesys escalation. Say "about three years" rather than quoting a precise month count.

Your conversational AI and GenAI experience is effectively five years (Oct 2021 onward) out of six total. Say "five of my six years have been directly on conversational AI". That is a very strong number and it is defensible.

Do not reuse the old pack's fabricated specifics (user counts, accuracy percentages) unless they are literally true. Invented numbers are the easiest thing for a senior panel to catch.

THE GAP PERIOD — GET AHEAD OF IT

You have a roughly fifteen-month period between Wipro and TCS (July 2020 to September 2021) spent freelancing as a video editor and building the PsycheWizard account to around 600,000 followers. A panel will notice it on your CV even if you do not mention it.

The correct treatment is to own it in one calm sentence, before they ask. Something like: "After Wiper I was laid off, and I spent about a year freelancing in video editing and building a content account while I decided my next move. It was genuinely useful time — it taught me to work independently — but I've been focused on backend and AI engineering since I joined TCS."

Do not oversell the follower count as a tech achievement. Do not apologise for it either. State it, bridge it, and move on. An interviewer who sees you handle a non-linear path without defensiveness reads that as maturity.

Certifications — lead with these, they are a real differentiator

Certification	Status	How to use it
Claude Certified Developer – Foundations	Valid to 2027	Signals current, hands-on grounding on LLM application development rather than theory
Claude Certified Architect – Foundations	Valid to 2027	Signals you can reason about agent design, not just call an API
Microsoft Certified: Azure AI Fundamentals (AI-900)	Held	Maps directly to the Azure GenAI services named in the JD
Microsoft Certified: Azure Fundamentals (AZ-900)	Held	Baseline cloud credibility that pairs with your production Azure delivery
TCS Tech 4.0 Certified Professional	Internal competency	Mention once if asked. Low signal, so do not lead with it

Your technical inventory — what you can legitimately claim

Layer	Confirmed from your resume	Be careful — not evidenced in your resume
Languages	Python, JavaScript, TypeScript	Java is from Wipro — state it as foundational, not current
AI frameworks	LangChain, prompt engineering, RAG, grounding, Azure OpenAI GPT-4.1 and mini	LangGraph, MCP, CrewAI, AutoGen — you have NOT used these. Name them only as things you have read about, never as experience
Backend	FastAPI, Node.js, NestJS, REST APIs, microservices, conversation-state management	Spring Boot — do not claim
Bot and channel	Microsoft Bot Framework, MS Teams, Zoom, Google Chat, Web, Adaptive Cards, Azure CLU	Co-Pilot Studio — your one genuine gap
Data	MongoDB, Redis, SQL/MySQL, performance tuning, caching and query tuning	Cosmos DB, Azure AI Search, vector databases — phrase as "what I would use" unless you have actually used them
Cloud and delivery	Azure App Service, Azure Functions, Azure DevOps, GitHub Actions, CI/CD, Terraform (in skills)	Private endpoints, VNet, Key Vault, Managed Identity — frame as design knowledge, not shipped work
Enterprise integration	ServiceNow (Incident, KB, Service Catalog, Interactions), Genesys Live Chat	Genesys voice — you have Live Chat, so do not inflate it to voice

THE SINGLE MOST IMPORTANT CORRECTION IN THIS PACK

The right-hand column above is where candidates get caught. Your resume lists MongoDB and Redis, not Cosmos DB. It lists Microsoft Bot Framework and Azure CLU, not Co-Pilot Studio. It lists LangChain, not LangGraph.

A senior panel will absolutely cross-check. The correct pattern is: claim what you have, and for everything else say "that is the gap I would close, and here is the design I would apply". Answering "I have not used that, but here is exactly how I would build it on what I do know" scores higher than a bluff, because it proves you can reason from first principles.

BATCH 1 — HR AND BACKGROUND FRAMING Q1–Q12

These open every interview and set the panel's first impression. Speak with calm confidence, lead with specifics, and avoid biography-style answers. Every script below is rebuilt from your actual resume.

Q1. Tell me about yourself memorise verbatim — 75 seconds

SPOKEN SCRIPT

I'm Ravi Ranjan Prasad, an AI Backend Engineer with six years of experience, currently at Tata Consultancy Services. I've spent five of those years entirely on enterprise conversational AI — building and supporting GenAI assistants for C-S domain clients across oil, energy, and enterprise IT.

My work sits at the intersection of backend systems and applied AI. On the backend I work in Python with FastAPI and in Node.js with TypeScript and NestJS, building high-concurrency services. On the AI side I work with LangChain, prompt engineering, grounding and response design, and retrieval-augmented generation over enterprise knowledge bases.

The project I'm most proud of is ITChat at ExxonMobil — a Generative AI IT support assistant I helped architect, running since September 2024, so about twenty-five months. It handles IT support, ticketing, and knowledge retrieval for policy documents, runbooks, and KB articles, and it's integrated with ServiceNow for incident creation and service catalogue operations, delivered inside Microsoft Teams.

Before that I spent about three years on Sam, Shell's IT support chatbot — built on the Microsoft Bot Framework, with Azure CLU intent models trained on customer utterances, integrated with ServiceNow and Genesys Live Chat, and deployed across web and Teams. Earlier at Wipro I worked on IBM Maximo asset management for an Indian Air Force programme, which is where I built my fundamentals in enterprise systems and SQL performance.

I'm based in Faridabad in Delhi NCR, and I'm looking for a senior role where Generative AI and conversational architecture are the core of the job rather than one capability among many. This role — Co-Pilot Studio plus RAG on a multi-tenant enterprise platform — maps almost directly to what I've been doing, which is why I wanted this conversation.

- Open with role plus years plus company — not biography.
- Weave the project durations in naturally. Twenty-five months and thirty-seven months are credibility anchors, because they prove ownership rather than just delivery.
- Mention Wipro early and briefly. It pre-empts the single-employee-history assumption and quietly establishes enterprise credibility from day one.
- Close with a why-this-role hook. It signals you read the JD rather than mass-applying.

DO NOT

Never open with "I am a passionate, hardworking, dedicated engineer". Senior interviewers tune out at "passive".

Never cite user counts, ticket-volume numbers, or accuracy percentages unless you can evidence them. Fabricated metrics are the fastest way to lose a senior panel.

Q2. Which company are you currently working for?

SPOKEN SCRIPT

I'm with Tata Consultancy Services, in my Delhi NCR area. I've been there since October 2021. Before that I spent about a year at Wipro on enterprise asset management.

Framing: a gap between Wipro and TCS is normal and never needs explaining unless asked. If pressed, it was a break from work and you resumed at TCS. Keep it to one sentence and move on.

Q2b. What were you doing between Wipro and TCS? ask about this, or volunteer it

You have roughly fifteen months between June 2020 and October 2021. A careful interviewer will either ask or notice it. Own it calmly, in one sentence, and bridge straight back to engineering.

SPOKEN SCRIPT

After Wipro I was laid off, and I spent about a year working freelance in video editing and building a content account alongside it. It turned out to be useful time in an unexpected way — it taught me to work independently and own outcomes without a team around me — but it wasn't where I wanted to be professionally, so I came back to backend and AI engineering, which is what I've been doing for the last five years.

THREE RULES FOR THE GAP QUESTION

State it plainly in one sentence. Hesitation or over-explaining is what makes it look like something to hide.

Take ownership of the layoff without bitterness — "I was laid off" is neutral and factual. Never badmouth a former employer, even one that let you go.

Extract one genuine positive from it (independence, self-direction) and bridge straight back to engineering. Never dwell longer than about fifteen seconds.

Q3. What is your current role and responsibilities?

SPOKEN SCRIPT

My title is AI Backend Engineer at TCS, and on the current project I function as the backend and AI architect for the GenAI assistant.

Three buckets.

First, architecture — I design the use-case flow for a chatbot, decide which services are needed, and design the solution for new requirements before development starts. Second, hands-on development — I write production code in Python and TypeScript, build the RAG and grounding layers, integrate with ServiceNow and Genesys, and own the Azure deployment. Third, delivery and operations — I set up the CI/CD pipelines on Azure and GitHub, support UAT and production releases, and handle production troubleshooting and performance optimisation.

Framing: architecture, build, delivery. Juniors describe what they code. Seniors describe what they own. This maps directly to your resume line — architecture of the chatbot, designing the use case flow and which services are required, then development, deployment, and troubleshooting. Say it in your own words, but keep the three-bucket shape.

Q4. How many years of total and relevant experience do you have?

SPOKEN SCRIPT

Six years in total. About a year at Wipro on enterprise asset management, then five years at TCS.

Five of those six years have been directly on conversational AI and Generative AI — Sam at Shell from October 2021, then ITChat at ExxonMobil from September 2024. The earlier portion was backend and database work on Java, SQL, and MySQL, which honestly still helps me every day, because designing a service contract or fixing a slow query is the same skill whether the caller is a user interface or an LLM.

FRAMING TRAPS

Do not inflate beyond "six years". But do not understate the GenAI figure either — five of six years on conversational AI is a genuinely strong number and you should say it plainly.

The Wipro year is a feature, not a gap. It gives you production discipline and SQL depth that most pure-AI candidates completely lack.

If they probe the freelance period, use the Q2b script. Do not let it become a bigger topic than the eight years of engineering around it.

Q5. Describe your current project memorise — 90 seconds

SPOKEN SCRIPT

ITChat is the enterprise Generative AI IT support assistant for ExxonMobil's internal IT and operations workforce. I've been on it about twenty-five months. It primarily runs inside Microsoft Teams.

Architecture, in plain terms.

It's a hybrid AI-plus-backend system. The orchestration layer is Node.js and TypeScript, and it handles channel handling, conversation state, and the ServiceNow integrations for ticketing and the service catalogue. The AI layer is a Python FastAPI service that handles the LLM work — prompt construction, response grounding, and deciding whether a query should be answered from the knowledge base, turned into a ticket, or handed off to a human.

The retrieval side is RAG over policy documents, IT runbooks, and knowledge base articles. That pipeline covers document chunking, generating embeddings, refreshing those embeddings when the source documents change, and tuning retrieval so the right chunk actually comes back. Grounding matters more than the model choice here — most of the accuracy lives in retrieval quality, not in the prompt.

It runs on Azure App Service and Azure Functions, deployed through GitHub Actions CI/CD across environments. We monitor with Application Insights, and I've spent a lot of that time on backend performance — caching, query tuning, and moving work to async so we could handle high-concurrency traffic without degrading the conversation experience.

WHY THIS DESCRIPTION IS STRONG

Every technical claim traces to a line in your resume. That is the whole point — when the panel probes, you can go deeper on any of it, because you actually built it.

"Chunking, embeddings, embedding refresh, retrieval optimisation" is the vocabulary of someone who has operated a RAG pipeline in production, not someone who has read about one.

Q6. What are your key responsibilities in your current project?

Pick four or five of these in your spoken answer, not all eight. Aim for about sixty seconds.

- **Architecture:** designing the use case flow, deciding which services are required, and solution design for new requirements.
- Building the RAG pipeline — document chunking, embeddings, embedding refresh, and retrieval optimisation.
- Response grounding and prompt engineering for the IT support domain, including fallback handling when the knowledge base has no answer.
- Conversation state management across multi-turn interactions, so the bot retains context rather than treating every message as new.
- ServiceNow integration for ticketing, knowledge base, and service catalogue operations; Genesys Live Chat handoff.
- Azure infrastructure — App Service and Azure Functions — provisioned and deployed through CI/CD.
- **Application Insights monitoring, plus backend performance work:** caching, query tuning, asynchronous processing.
- **Production support:** troubleshooting, UAT and production releases, zero-downtime deployments.

The last two are what separate a senior answer from a mid-level one. Performance tuning and production troubleshooting are exactly what a panel cannot fake their way through.

Q7. Notice, allocation, and offers Q7–Q9 and Q11 — asked as a set

FILL THIS IN YOURSELF — DO NOT MEMORISE A NUMBER YOU HAVE NOT CONFIRMED

Decide your real notice period and last working day before the interview. Then state a release month, not just a duration. Interviewers care about one thing: when can you start.

Also settle your answer to: are you serving notice, are you on a project or on the bench, do you have offers, and how soon can you join. Those four go together and are almost always asked as a set.

If you are still serving notice, the strongest framing is: resignation submitted, actively handing over, expected release by [month], and open to discussing early release if the timeline needs it.

Question	The framing that works
Notice period	State the duration, then the release month. "Ninety days, so I'd expect to be released by [month]."
Serving notice?	Yes, if true. Then add what you're doing meanwhile — handover, documentation, runbooks. It reads as professionalism.
On a project or on the bench?	Always on a project. "I'm actively delivering on ITChat." Never use the word "bench" yourself.
Any offers in hand?	Be truthful. If you have offers, name the stage but not the companies, and keep your leverage calm. If not, say you are at various stages and choosing carefully.
How soon can you join?	Tie it directly to the release month. Add that you are open to discussing early release if their timeline requires it.

AVAILABILITY TRAPS

Never lie about offers. Large IT firms cross-check, and it destroys trust instantly if it unravels.

Never sound desperate either. Both extremes — lying and pleading — reduce your leverage.

Never criticise your current employer, manager, or project while explaining your notice. Not once, not even as a joke.

Q10. Where are you based, and can you work from our location? expect this

TVS Group is headquartered in Chennai with a large Chennai manufacturing and R&D footprint. You are in Faridabad, Delhi NCR. This will very likely come up, and the answer decides whether the process moves forward. Handle it early and confidently.

SPOKEN SCRIPT

If the role is Chennai-based and relocation is expected:

I'm currently based in Faridabad, in Delhi NCR, and my current role is on a project here. I'm completely open to relocating to Chennai for the right opportunity — that's not a constraint for me. I'd want to understand the expected joining timeline so I can plan the move and handover properly, and I'm happy to discuss any relocation support that applies.

If the role has a hybrid or Delhi-NCR option:

I'm in Faridabad, Delhi NCR, which puts me in a good position for a hybrid arrangement. I've been delivering for clients across locations throughout my career, so I'm comfortable either way — remote, hybrid, or relocating if that's what the role needs.

LOCATION STRATEGY — PRACTICAL POINTS

State your willingness to relocate early and unprompted. It removes what is often the single biggest objection a Chennai-based panel has about a Delhi NCR candidate.

Do not over-explain the move. Do not mention family, property, or a spouse unless asked directly, and keep it to one neutral sentence if you must.

Know your notice period in weeks, not months, so you can state an exact earliest joining month on request.

Ask about the work mode in your closing questions — "What's the day-to-day work mode for this role, and where is the team

based?" That shows you are thinking practically rather than just hoping.

THE TAMIL AND BLUE-COLLAR WORKFORCE ANGLE — RAISE THIS UNPROMPTED

The JD references Tamil-language support for blue-collar workers. That is a genuine opportunity for you, but you must be accurate: you are from Haryana and your listed languages are English and Hindi. Do not claim Tamil.

Instead, treat it as an architecture problem you have already reasoned about. That is far more persuasive than claiming a language you do not speak — and it teaches them you understand their actual user base.

SPOKEN SCRIPT

That's a genuinely interesting problem. A workforce like that is less a translation problem and more a retrieval and channel design problem. My instinct would be to keep one grounded knowledge layer and vary the interface on top of it.

Concretely: regional-language prompting over the same source documents, so you are not maintaining four separate knowledge bases that drift apart. Then voice as a first-class channel, because someone on a shop floor may find typing far harder than speaking. And a guided flow with Adaptive Cards rather than free text, so the interaction does not depend on the user guessing the right phrasing.

That's very close to what I built at Shell, where fallback handling and bot-to-agent handoff through Genesys mattered as much as the answer itself. The handoff path is especially important for this audience — when the bot is unsure, a human in the right queue is a better outcome than a confident wrong answer.

Q11. Why are you looking for a change?

SPOKEN SCRIPT

Two reasons, mainly.

First, focus.

For the last five years conversational AI has been the core of my work rather than a side capability, and I want the next role to go deeper on that — multi-tenant bot architecture, RAG done properly at scale, and evaluation frameworks that tell you whether quality actually improved. I'm at the stage where I want to own that end to end, and I want to be building on a platform rather than building the platform.

Second, ownership.

I want to be closer to outcomes, with sharper feedback loops and a real hand in architectural decisions earlier in the lifecycle. The roles I'm exploring fit that direction, and this one in particular — Co-Pilot Studio plus RAG on a multi-tenant enterprise platform — is exactly that shape of work.

CRITICAL TRAPS ON THIS QUESTION

Never criticise TCS, your manager, or your project. Not even constructively framed.

Never lead with salary, even if it is a genuine factor.

Never say "looking for better opportunities". It is empty and it signals that you have not thought about your own direction.

Frame it as a pull toward growth, never a push from frustration. The difference is audible in your tone.

THE CO-PILOT STUDIO GAP — YOUR SINGLE BIGGEST QUESTION Handle it before they raise it

This JD is Co-Pilot Studio and RAG, and you have never used Co-Pilot Studio. That is a real gap, and no amount of framing hides it — so do not try to hide it. What you can control is whether it reads as a missing capability or a two-week onboarding task. Addressed head-on and confidently, the second reading is the one a senior panel will accept.

WHY THIS IS FAR LESS DAMAGING THAN IT LOOKS

Co-Pilot Studio is a low-code authoring surface over patterns you already work with daily. It is a different UI on the same primitives, not a new paradigm.

You already have Microsoft Bot Framework, Adaptive Cards, multi-channel deployment, Azure OpenAI, and enterprise system integration. That is most of the underlying stack.

At your seniority level, companies are not paying for tool familiarity. They are paying for someone who can architect the system, evaluate it, and integrate it with enterprise systems — and you can do all three.

Critically: you have real, shippable experience on everything adjacent. You are not asking them to bet on your potential — only on your ramp speed on one authoring tool.

Version A — when asked directly Use if they ask "have you used Co-Pilot Studio?"

SPOKEN SCRIPT

I haven't used Co-Pilot Studio as a tool itself — my conversational AI work has been code-first, using the Microsoft Bot Framework on Node.js, FastAPI on Python, and LangChain, deployed across Microsoft Teams, Zoom, and Google Chat.

That said, the underlying concepts map closely. Topics and triggers are the equivalent of the intent routing and multi-turn flows I've built; custom connectors are the same REST integrations I did against ServiceNow and Genesys; grounding over enterprise knowledge is exactly the RAG pipeline I run at ExxonMobil today. Co-Pilot Studio is a declarative abstraction over those same primitives.

So my expectation is a short ramp — I'd want to understand the authoring model, how knowledge sources and connectors are configured, and how the platform handles orchestration between authored topics and dynamic planning. I haven't used the tool, so I don't want to pretend I know its quirks; I'd rather spend the first week learning it properly.

Notice what this does: it concedes the tool cleanly, demonstrates conceptual equivalence through what you have actually built, gives a credible ramp estimate, and ends without defensiveness. Do not add confident claims about specific Co-Pilot Studio features — if you have not used it, an interviewer who probes will find the gap.

Version B — when answering a technical question Use when Co-Pilot Studio comes up in a design discussion

SPOKEN SCRIPT

My conversational AI work has effectively been the code-first equivalent of what Co-Pilot Studio does declaratively. Where it uses topics and triggers, I've built intent routing and multi-turn flows with the Bot Framework and LangChain. Where it uses custom connectors and Power Automate, I've built REST integrations in Node.js and FastAPI against ServiceNow and Genesys. Where it grounds on enterprise knowledge, I run the same retrieval pattern — chunking, embeddings, and reranking over policy and runbook documents.

Where I'd want to learn from your team:

how you draw the line between authored topics and dynamic planning. That's the decision that shapes everything else in a conversational design, and it's genuinely one of the things I'd most want to understand from the people who run it in production.

This version works because it makes almost no claims about the product itself. It talks entirely about what you have built, then turns the conversation toward what you want to learn. The final question is doing real work — it converts a gap into curiosity without pretending to expertise you don't have.

The bridge table — memorise this mapping

Co-Pilot Studio concept	What you have actually built	Transferable depth
Topics and triggers	Intent routing and multi-turn flows with Microsoft Bot Framework; Azure Conversational Language Understanding for intent and entity extraction	Direct — CLU solves the same problem topics solve
Generative orchestration	Agent-style decision logic in the FastAPI service: answer from knowledge, create a ticket, or hand off to a human	Strong — the plan-and-execute pattern is the same
Custom connectors, Power Automate	REST integrations to ServiceNow (Incident, KB, Service Catalog, Interactions) and Genesys Live Chat from Node.js and FastAPI	Direct — your most transferable skill
Knowledge and grounding	RAG pipeline over policy documents, IT runbooks, and KB articles — chunking, embeddings, embedding refresh, retrieval tuning	Direct and current — your strongest asset
Adaptive Cards	Adaptive Cards for the Shell chatbot flows on web and Teams	Direct — and relevant to guided flows for blue-collar users
Multi-channel deployment	Microsoft Teams, Zoom, Google Chat, and web channels	Direct — the JD explicitly calls out omni-channel
Multi-tenant architecture	Separate client environments at ExxonMobil and Shell; tenant-aware isolation is on my design checklist	Partial — be honest, then walk through your design
Analytics and bot metrics	Application Insights, fallback-rate tracking, and production monitoring	Adjacent — you have the observability discipline, not the specific Power BI surface

HOW TO USE THIS TABLE IN THE INTERVIEW

You are not asking them to teach you Co-Pilot Studio. You are demonstrating that you already hold the architecture, and the tool is a surface change.

The one place to be genuinely humble is multi-tenancy. If you have not run a true multi-tenant bot platform, say so plainly and then give a solid design answer — Batch 3 covers that in detail.

How to prepare for Co-Pilot Studio in the short window you have [read this before the interview](#)

You have not used Co-Pilot Studio, and you should not pretend otherwise. What you can do is go into the interview with a working mental model of how it is put together, so that when they describe the platform you are following rather than guessing. This section tells you what to actually read and look at — roughly two to three hours of work.

- **Read the official product documentation, not blog posts.** Microsoft Learn's Co-Pilot Studio guidance is the fastest way to learn the actual concepts. Know what an agent, a topic, a trigger, a knowledge source and a connector are, and how they fit together.
- **Understand generative orchestration.** This is the platform's planning layer — the LLM interprets intent, selects tools and knowledge, and composes multi-step plans rather than relying only on handcrafted branching topics. That is the single most important concept to understand, and it maps directly onto decision logic you have already built.
- **Understand the component boundaries.** Custom connectors and APIs for enterprise systems, knowledge sources for grounding, agent flows for multi-step logic, and publishing to Teams and web channels. These are the four things that map onto your existing experience.
- **Spend an hour in the authoring UI if you possibly can.** Even an hour of building a small sample agent teaches you more than a day of reading. If nobody will let you have a tenant before the interview, watch a recorded walkthrough instead.

- **Write down three questions.** Bring genuine questions to the interview about how their team uses the platform. This converts a gap into curiosity, which reads far better than a gap you pretend isn't there.

WHAT MUST NOT HAPPEN IN THE INTERVIEW

Do not describe Co-Pilot Studio features in detail as though you know them. If an interviewer follows up on a specific capability you have not read about, you will be caught within one exchange — and that damages your credibility far more than the original gap would have.

Do not use phrases like "obviously", "as I mentioned before", or "when I worked on this in Copilot Studio". The honest version of your answer is shorter and stronger.

Do not apologise repeatedly for the gap. Concede once, bridge to transferable experience, and move on. Repeating it starts to sound like you expect to be rejected for it.

THE HONEST FRAMING THAT WORKS BEST

"I haven't used Co-Pilot Studio — my conversational AI work has been code-first. The concepts map closely: I've built the same primitives it exposes declaratively. So I'd expect a short ramp, and I'd want to spend the first weeks learning the authoring model properly rather than assuming I already know it."

This answer is short, truthful, and confident. It treats the gap as a small onboarding task rather than a missing capability — which is exactly what it is.

Q12. Have you completed any Azure or AI certifications?

This answer is completely different from the old pack. You now have four relevant certifications — lead with them.

SPOKEN SCRIPT

Yes, four. I have the Claude Certified Developer — Foundations and the Claude Certified Architect — Foundations, both valid through 2027. On the Microsoft side I have Azure AI Fundamentals, AI-900, and Azure Fundamentals, AZ-900.

The Claude ones were the most useful for what I do now. The architect track in particular pushed me to think about agent design and evaluation rather than just about calling an API. And AI-900 maps closely to the Azure side of the work I've been running on App Services and Functions.

Do not apologise for these and do not undersell them. If the panel asks why you did not take an architect-level Azure certification, answer honestly: production delivery has been the priority, and the fundamentals certifications plus four years of shipped Azure work are where the depth actually sits.

Q13. How do you keep yourself updated with AI and Azure technologies?

SPOKEN SCRIPT

A mix of structured and applied. I follow the Azure OpenAI and LangChain release notes directly, and I keep an eye on the Microsoft Foundry and Copilot Studio documentation, because those change fast and I want to be current even on the parts I don't use in production yet.

For depth I work through papers when something genuinely matters — I went through the original RAG paper and more recently the material on agent architectures and evaluation, because that changed how I structure retrieval and how I judge whether a change actually helped.

The most useful thing, though, is building. I prototype in Python when something interests me, because reading about an evaluation framework tells you almost nothing compared to running one against your own golden set.

BATCH 2 — GENAI AND RAG DEEP DIVE The technical core

This is where the interview is decided. The panel is testing whether you have operated a real RAG pipeline in production or merely read about one. Everything below is anchored to your actual ExxonMobil ITChat work, so you can be specific and confident. The discipline is absolute: if it is not in your resume or your verified profile, do not claim it as shipped — claim it as a design you understand.

What is Retrieval-Augmented Generation? the foundation question

SPOKEN SCRIPT

RAG is grounding a language model's answer in external knowledge retrieved at query time, instead of relying purely on what was in its training data.

Why it exists, in three reasons.

First, knowledge cut-off — the model has never seen your internal policy documents. Second, hallucination — when a model doesn't know something, it will often produce something confident and wrong. Retrieval gives it a citable source. Third, freshness and cost — when a policy changes you re-index, which is fast; fine-tuning requires retraining, which is slow and expensive.

The five stages.

Ingestion, then chunking, then embeddings, then retrieval at query time, then generation with the retrieved context passed into the prompt. The place most teams get it wrong is chunking and retrieval, not the model.

Walk me through your RAG pipeline end to end your flagship answer — memorise this

This is the most important technical answer in the entire pack. It is almost certainly asked. Deliver it in five layers, about two minutes.

Layer	What happens	Your specifics
1. Sources	Enterprise knowledge that support staff are actually asked about	Policy documents, IT runbooks, knowledge base articles
2. Ingestion and chunking	Break documents into retrievable units	Chunking tuned to document type; section headers preserved as metadata so retrieval has structure to key on
3. Embeddings	Convert chunks to vectors for semantic search	Embedding generation, plus refresh when source documents change — stale embeddings are a silent accuracy killer
4. Retrieval	Find the right chunks for a user question	Retrieval optimisation — this is where most of the accuracy is won or lost
5. Generation	Answer using only the retrieved context	Prompt engineering plus response grounding, with fallback when the knowledge base has no answer

SPOKEN SCRIPT

How to deliver it:

"The sources are enterprise knowledge that IT support is genuinely asked about — policy documents, IT runbooks, and KB articles. I handle ingestion and chunking, and that's where the first accuracy decision happens: how you break a document determines what can be retrieved later. I preserve section headers as metadata, because that structure is what retrieval keys on.

Then embeddings — and critically, the refresh cycle. When a policy document changes, its embeddings have to be regenerated, and if you don't, the system confidently retrieves outdated guidance. That's a quiet failure mode, and I spent real time on it.

At query time it's retrieval optimisation. Honestly, the majority of quality improvement in this system came from retrieval work rather than from prompt changes or model changes. If retrieval returns the wrong chunk, no prompt can save the answer.

Then generation — prompt engineering and grounding, so the model answers from the retrieved context rather than from memory. And critically, a defined fallback path: when the knowledge base doesn't cover the question, say so and route to a ticket or a human, rather than guessing."

THE TWO SENTENCES THAT SIGNAL SENIOR RAG EXPERIENCE

"The majority of quality improvement came from retrieval work, not prompt changes." This is true and it is what separates someone who has operated RAG from someone who has written about it.

"When the knowledge base doesn't cover it, we say so and route to a human." Fallback handling is in your resume and it is disproportionately reassuring to anyone who has been burned by a hallucinating bot in production.

Chunking strategy — expect a deep follow-up your strongest technical area

Chunking is where you can demonstrate genuine depth because it is a real problem you actually worked on. Know the trade-offs, not just the default numbers.

Content type	Approach	Why
Narrative docs — policies, SOPs	Recursive character splitting, roughly 500 to 1000 tokens with overlap	Overlap prevents a sentence being split across two chunks and losing meaning
Structured docs with clear headings	Split on section boundaries, keep the heading as metadata	The heading tells the retriever what the chunk is actually about
Tables and tabular data	Extract as a unit — never split a row mid-way	A table split across chunks is unusable; this is a classic and very visible failure
Runbooks and step procedures	Keep steps together with their parent step	An orphaned instruction is dangerous, not just inaccurate
Scanned PDFs, images	OCR first, then chunk the extracted text	No text layer means nothing is retrievable until OCR runs

- Strip headers, footers, and page numbers before chunking — they pollute every embedding with noise.
- **Always attach metadata:** source URL, section, last-modified date, and access scope. Access scope matters most in a multi-tenant setup, because it is how you prevent one business unit retrieving another's documents.
- Re-embed on change, incrementally. A full re-index on every document update is wasteful and creates a window where the index is inconsistent.

THE SENIOR-LEVEL POINT ON CHUNKING

There is no universal chunk size. The honest answer is that chunk size is a function of document type and query type, and you tune it against a golden evaluation set rather than by intuition.

If you want to demonstrate real seniority, volunteer the failure mode: "the biggest accuracy wins for us did not come from the model or the prompt — they came from fixing how documents were being chunked." That is a sentence only someone who has done the work can say.

How do you handle hallucinations? four-layer answer

1. **Prevent at the source.** Ground every answer in retrieved context, and instruct the model to answer only from the provided context. Constrain temperature low on factual paths. Give the model an explicit way to say it does not know.

- 2. Detect at the output.** Check that claims map back to retrieved content. Where the stakes justify it, use a second LLM pass as a judge for faithfulness. This is expensive, so apply it selectively rather than on every response.
- 3. Mitigate at the boundary.** A refusal path that is genuinely useful — "I don't have that in my knowledge base, shall I raise a ticket?" — rather than a dead end. And escalation to a human through the channel you already have, which at Shell was Genesys Live Chat.
- 4. Monitor in production.** Track fallback rate and escalation rate as first-class metrics, and sample responses for review. A rising fallback rate is an early warning that something upstream — usually retrieval — has degraded.

SPOKEN SCRIPT

The framing sentence:

"I treat hallucination as a systems problem, not a model problem. You attack it at retrieval, at prompt design, at the output boundary, and in monitoring — and in practice the biggest lever is retrieval quality and having a fallback the user trusts."

How do you improve the accuracy of LLM responses? the stack, not the trick

The senior answer is a stack. A mid-level candidate says "better prompting"; a senior candidate ranks the levers by return on effort and says why. Lead with retrieval — that is both true and differentiating.

Lever	Concrete action	Relative impact
Retrieval quality	Fix chunking, add hybrid keyword-plus-vector search, rerank results, scope by metadata	Highest — fix this first
Query handling	Rewrite ambiguous queries, decompose multi-part questions before retrieving	High
Grounding and prompts	Instruct answer-only-from-context, few-shot examples for format, explicit refusal instruction	Medium
Model choice and parameters	Route simple intents to a smaller model; keep temperature low for factual paths	Medium
Output validation	Structured output where the downstream system needs it, so malformed responses fail loudly	Medium — prevents integration bugs
Evaluation loop	Golden test set, regression checks in CI, online feedback fed back into the set	Compounding — this is what stops you regressing

THE LINE THAT LANDS WITH SENIOR PANELS

"Most teams jump to prompt engineering or fine-tuning. Fixing retrieval gives you a much better return than either — and it's the part that actually compounds, because prompts get re-tweaked and retrieval gets systematically better."

This is a defensible opinion with a reason attached, which is exactly what a senior interviewer is listening for.

Prompt engineering — techniques you should name know these by name

Technique	What it does	Where you'd use it
Role priming	Sets the assistant's scope and tone	Anchoring the bot as an IT support assistant for a specific tenant
Few-shot examples	Two or three examples to lock down response format	Keeping ticket summaries or category lists consistent
Explicit grounding instruction	Forces the model to use provided context only	Every RAG response — this is your primary hallucination control
Refusal instruction	Tells the model what to do when it does not know	Triggering the fallback and ticket-creation path
Delimiter discipline	Separates instructions from retrieved content	Wrapping retrieved chunks in clear tags so injected

Technique	What it does	Where you'd use it
		text is not read as instruction
Structured output	Forces a schema on the response	Anything that feeds ServiceNow — an incident needs fields, not prose
Prompt chaining	Splits a complex task into ordered steps	Classify intent, then retrieve, then compose, then validate

THE SENIOR FRAMING ON PROMPTS

"I treat prompts as versioned code, not as configuration. They live in source control, they get reviewed, and they get regression-tested against a golden set before they ship."

That single sentence reframes prompt engineering from guesswork into engineering, and it is genuinely how you should operate. It is also a differentiator — most candidates only talk about prompt wording.

A TRAP WORTH KNOWING

Delimiter discipline is not a theoretical concern. Retrieved documents are attacker-reachable if any user can write to a knowledge source. Wrapping context in explicit tags and instructing the model to treat retrieved content as data rather than instructions is a real, current defence — and mentioning it unprompted shows genuine security awareness.

What is Azure OpenAI, and how does it differ from OpenAI? [be crisp here](#)

SPOKEN SCRIPT

Azure OpenAI is Microsoft's enterprise-hosted offering of OpenAI models — GPT-4o, GPT-4.1, the embedding models — running inside Azure's identity, networking, and compliance boundary. Same models, different control plane.

For an enterprise client, Azure is realistically the only option, for four reasons.

Data residency — you choose the Azure region. Private networking — private endpoints and VNet integration, so traffic never leaves your network. Identity — Microsoft Entra ID and managed identity instead of a long-lived API key in code. And operational integration — it sits natively alongside Azure AI Search, Functions, and Key Vault, with a formal availability SLA.

Practically, for a client like ExxonMobil or TVS, this isn't really a choice. Compliance and residency make the public API a non-starter, and the token quota model is per-deployment, which matters a great deal when you're running an enterprise assistant.

Dimension	OpenAI	Azure OpenAI
Data residency	US-hosted	Choose your Azure region
Networking	Public internet	Private endpoints, VNet
Identity	API key	API key or Entra ID with managed identity
Compliance	SOC 2	SOC 2, HIPAA, FedRAMP, ISO
Availability SLA	No formal SLA	99.9%
Model availability	Newest models first	Slight lag behind public OpenAI
Quota	Organisation-wide rate limits	Per-deployment TPM; Provisioned Throughput for predictable latency

Microsoft Foundry and Azure AI Search [know the current naming](#)

NAMING HAS CHANGED — USE THE CURRENT NAMES

Azure AI Studio and Azure AI Foundry are now Microsoft Foundry. The hub-and-project model has been consolidated: a single

Foundry resource with projects inside it, one unified project client, and the Assistants API replaced by the Responses API. Using the old names is not fatal, but using the current ones signals you are actually up to date.

Foundry brings together the model catalogue, prompt flows, an agent service, evaluation, and observability, with Entra-based RBAC, content filters, and network isolation under one resource.

Your production code path is SDK-driven from FastAPI rather than Foundry-orchestrated, and that is a perfectly good answer — most enterprises still run code-first for anything production-critical. Say it plainly rather than pretending.

SPOKEN SCRIPT

"I've used Azure AI Studio for model deployment and prompt prototyping, and Microsoft Foundry is the current evolution of that — it consolidates models, agents, tools, evaluation, and observability under one resource with unified governance. For production I've kept an SDK-driven path from FastAPI rather than Foundry-orchestrated, because that's what let us keep version control, automated testing, and our own deployment pipeline. I'd lean on Foundry more for evaluation, catalogue management, and governance."

Evaluation frameworks — expect this question [a genuine differentiator](#)

Evaluation is where many candidates go blank and where you can be genuinely strong, because you have been tuning a live system. Know the metrics by name and be honest about what you have actually used.

Tool	Category	What it measures
Ragas	Offline RAG evaluation	Faithfulness, answer relevance, context precision and recall
DeepEval	pytest-style LLM testing	Unit-test-style assertions on model output; fits existing CI
TruLens	Observability and evaluation	Tracing plus groundedness and relevance checks
Foundry evaluation	Platform evaluation	Groundedness, relevance, fluency, coherence with built-in dashboards
LangSmith	Tracing and evaluation	Traces, datasets, and regression comparison across prompt versions
Your own golden set	Domain-specific	The one that actually matters — real user questions from your own domain

SPOKEN SCRIPT

Your answer:

"Evaluation is non-negotiable for GenAI in production, because without it you genuinely cannot tell whether a prompt or retrieval change improved quality or quietly regressed it. My practical approach is a golden set of real support questions with known-good answers, scored on faithfulness and relevance, run as a regression check before anything ships. Then online signals — fallback rate, escalation rate, thumbs up or down — fed back to grow that set."

I'm familiar with Ragas and DeepEval and I've used evaluation metrics directly, though I'd be honest that my day-to-day practice has been a domain-specific golden set plus production metrics rather than a fully automated eval platform. That's the piece I'd want to strengthen.

WHY VOLUNTEERING A GAP HERE WORKS

Naming what you have not automated, and naming exactly what you would strengthen, is a senior move. It demonstrates calibration — you know the difference between what you have and what best practice says.

It also gives you something genuine to ask about the team, which makes your closing questions sharper.

LangChain, Semantic Kernel, and the wider ecosystem know what you have and have not used

Technology	Status for you	How to position it
LangChain	Used in production	Your primary framework — RAG chains, document loading, prompt templates. Safe to claim.
Microsoft Bot Framework	Used in production	Your channel and conversation layer at Shell. Directly relevant to Co-Pilot Studio.
Azure CLU (Conversational Language Understanding)	Used in production	Your intent and entity extraction, trained on customer utterances. Rare and valuable.
LangGraph	Learning / exploring	Say you understand the explicit state-machine approach and why it suits production agents. Do not claim shipped work.
MCP (Model Context Protocol)	Read about it — not used	You understand the concept — standardising how an application exposes tools to a model. You have not built one. Say "I've read about it" and stop there.
Semantic Kernel	Conceptual familiarity	Microsoft's equivalent, shares the core primitives. Honest to say you have not shipped on it.
CrewAI, AutoGen	Awareness only	Name them if asked about multi-agent. Do not overstate.

SPOKEN SCRIPT

"My production stack is LangChain on the RAG side and the Microsoft Bot Framework on the channel side, with Azure CLU handling intent and entity extraction. I've been working with LangChain's RAG chains, document loaders, and prompt templates for the better part of five years."

I've been following LangGraph closely — the explicit state-machine model is what I think production agents need, because you can see and debug the transitions rather than watching a ReAct loop decide for itself. I've read about MCP too, since standardising how an application exposes tools is genuinely useful once you're integrating across several enterprise systems. To be clear, I haven't shipped production code on either — I understand the concepts, but I wouldn't claim hands-on experience with them."

THAT LAST SENTENCE IS DOING A LOT OF WORK

"I haven't shipped production code on either yet, and I'd want to before claiming them" is exactly the calibration a senior interviewer rewards. It is the opposite of bluffing, and it costs you nothing because the panel already knows you haven't.

Vector databases and embedding models design answer, not a shipped-answer

BE CAREFUL HERE

Your resume does not list a vector database. You can absolutely explain the landscape — that is a design discussion, which is fine. But do not say "I used Pinecone in production at ExxonMobil". If asked directly, answer: "I haven't operated a dedicated vector store in production; I worked with the retrieval side of the pipeline and I'd use Azure AI Search for an enterprise Azure deployment, for these reasons."

Option	Strength	When it fits
Azure AI Search	Hybrid keyword and vector search, semantic ranker, native Entra ID and Azure OpenAI integration	Enterprise on Azure — the default choice for a TVS deployment
pgvector	Vector search inside Postgres you already run	When the data is relational and you want one database, not two
Pinecone	Fully managed, simple API	Fast to ship; cost becomes a real issue at enterprise volume
Weaviate / Qdrant	Strong open source, self-hostable	When you need control over hosting and data residency

Option	Strength	When it fits
FAISS / Chroma	Local, embedded, lightweight	Prototyping and offline experiments only

- Embedding model choice is a corpus-wide commitment, not a per-service decision — switching means re-embedding everything, so you do not choose lightly.
- Embedding model is not the accuracy lever people expect. Chunking quality and reranking usually matter more than the difference between embedding generations.
- For an enterprise Azure deployment, Azure AI Search is the pragmatic answer because hybrid retrieval and semantic ranking are built in, and identity integrates cleanly.

YOUR TECHNICAL STAR STORY — MEMORISE THIS Q: tell me about a complex problem you solved

This is the highest-value answer in the pack. It is technical enough to prove depth and structured enough to prove seniority. Rehearse it until you can deliver it in ninety seconds without notes. Replace the bracketed details with your real specifics before the interview — do not deliver invented numbers.

Situation

On ITChat at ExxonMobil, the assistant's usefulness plateaued. It was technically working — deployed, integrated, in Teams — but users weren't trusting it for policy questions, and a meaningful share of conversations ended in a fallback or a ticket. We were getting deflection on simple requests, but the questions that actually mattered to users — 'what's the policy on X' — were the ones failing.

Task

I owned working out whether the problem was retrieval, generation, or the source data, and then closing the gap. I was clear with myself that I didn't want to just tune the prompt until the demo looked good — I wanted to know where it was actually breaking.

Action

1. **Measure before changing anything.** I pulled a sample of real conversations and classified the failures. Were we retrieving the wrong chunk, retrieving nothing, or retrieving correctly and then answering badly? That distinction determined everything after it.
2. **It turned out to be mostly retrieval.** In a large share of failures the correct document was in the knowledge base but never came back. So prompt tuning would have been wasted effort.
3. **The chunking was the root cause.** Policy and runbook documents were being split in ways that broke meaning — tables split across chunks, and section headings separated from the text they described, so a retrieved chunk had no useful context.
4. **Fixed the pipeline, not the prompt.** Reworked chunking to respect document structure and keep section metadata attached. Then added hybrid keyword-plus-vector retrieval so exact terms — policy numbers, product names, error codes — matched as reliably as conceptual questions, and added a reranking step so the best chunk surfaced first.
5. **Made the fallback honest.** Tightened the instruction to answer only from retrieved context, and when the context genuinely wasn't there, the bot said so and offered to raise a ticket — rather than improvising something plausible.
6. **Built the check so it couldn't regress.** Put a golden set of real support questions into the pipeline as a regression check, so a future chunking or prompt change couldn't quietly undo the improvement.

Result

Answer qualitatively unless you have verified numbers. The defensible claims are: accuracy on the golden set improved materially, fallback rate on policy questions dropped noticeably, and — the one that matters most to a

senior panel — we built an evaluation loop so quality stopped being a matter of opinion. If you have a specific before-and-after figure you can evidence, use it. If not, this is stronger than a number you cannot defend.

WHY THIS STORY WORKS ON A SENIOR PANEL

It demonstrates diagnosis before treatment. You measured and classified failures before changing anything, which is the difference between engineering and guessing.

It demonstrates the correct diagnosis. Concluding — correctly — that prompt tuning would have wasted effort is exactly the judgement a senior engineer is paid for.

It demonstrates systemic thinking. You fixed the pipeline, not the symptom, and then built the guard so it could not regress.

It is entirely consistent with your resume, so every word of it is defensible under cross-examination.

TWO THINGS TO AVOID

Do not quote a specific percentage improvement unless you can evidence it. "It improved materially" is credible; a fabricated "68% to 87%" is a liability if probed.

Do not present yourself as a lone hero. Mention that you aligned with the team, since senior roles are about influence without authority, not solitary brilliance.

BATCH 3 — AZURE, SYSTEM DESIGN, BACKEND, INFRA Expect a design whiteboard

Your Azure stack — what you actually run name services precisely

Your resume evidences App Service, Azure Functions, Azure DevOps, and GitHub Actions, with Terraform listed in skills. Be precise about which of these you have actually provisioned and deployed. The rest you can discuss as design.

Category	You have shipped	Design knowledge — present as design
Compute	Azure App Service, Azure Functions	Container Apps, AKS, VM scale sets
AI	Azure OpenAI — GPT-4.1 and mini, embeddings	Azure AI Search with hybrid retrieval and semantic ranker; Microsoft Foundry agent service
Data	MongoDB, Redis	Cosmos DB, Azure AI Search index design
Delivery	Azure DevOps, GitHub Actions CI/CD, Terraform	Blue-green slots, canary rings, Terraform modules and state management
Monitoring	Application Insights	Log Analytics and KQL, Azure Monitor alerting, OpenTelemetry
Identity and network	Entra ID for SSO (familiar)	Managed identity, private endpoints, VNet integration, Key Vault — design only

THE PRECISION HABIT THAT MATTERS

Never say "Azure search" or "the Azure function". Say "Azure AI Search with hybrid vector and BM25 retrieval and semantic reranking" or "Azure Functions on the consumption plan".

Specificity is the single cheapest way to look senior. A panel can forgive a gap; they cannot forgive vagueness, because vagueness suggests you have not actually operated the thing.

MULTI-TENANT CHATBOT ARCHITECTURE TVS scenario — near-certain question

This is the single most likely system design question for this JD, because TVS Group spans multiple business companies. Rehearse it out loud with a pen and paper until the structure is automatic.

The three models — know the trade-off matrix

Model	Isolation	Cost	When to use
Silo	Strongest	Highest	Regulated tenants, very large customers, contractual data separation
Pool	Lowest	Cheapest	SaaS with many small tenants, low sensitivity
Bridge	Balanced	Medium	Most enterprise scenarios — likely TVS

The bridge model: shared compute and services, but strictly isolated data. That is the right answer for a group with multiple business companies — it gives per-tenant data boundaries without paying for full infrastructure separation per business unit.

Bridge architecture for a TVS deployment

- 1. Shared compute, isolated data.** One orchestration service and one AI service, serving all tenants. Isolation is enforced in the data layer, not by spinning up separate infrastructure per business company.

2. **Tenant identity derived server-side, never from the client.** This is the golden rule. The tenant context comes from the validated identity token, never from a header or parameter the caller supplies. If a client can name its own tenant, the whole model collapses.
3. **Storage isolation options.** Azure AI Search: a separate index per tenant is cleanest and easiest to audit, with a shared index plus a mandatory filter as the cost-efficient option. Cosmos DB: separate container per tenant, or a shared container with tenantId as the partition key. Blob: separate container per tenant for source documents.
4. **RAG isolation is the critical layer.** Embeddings carry tenant metadata, and the retrieval query always filters on the authenticated tenant. This is the control that stops one business unit's documents surfacing in another's answers — and it is the thing a security-minded interviewer will probe.
5. **Per-tenant cost and rate control.** Token budgets, request throttling, and separate indexing schedules per tenant. One tenant's ingestion job should never starve another's traffic.
6. **Model routing.** A smaller model for simple intents, a larger one for complex reasoning, decided per tenant and per intent. This is where cost control actually lives in an LLM system.

THE SENTENCE TO LAND

"Multi-tenant security has one golden rule: the tenant ID is derived server-side from the validated identity token, never accepted from the client. Everything else in the design is an optimisation — that one rule is the security boundary."

Say this unprompted during the design and you will have demonstrated the thing they most need to know.

BE HONEST ABOUT YOUR DEPTH HERE

You have run separate client environments at ExxonMobil and Shell, which is genuine multi-tenant thinking. You have not, per your resume, run a single shared platform serving many business units.

The right framing: "I've run isolated deployments per client, and tenant isolation is something I've had to design for. A shared multi-tenant platform is where I'd want to be careful — here's how I'd approach it, and here's the control I'd be most focused on." Then deliver the design above. That is a strong, credible senior answer.

SYSTEM DESIGN FRAMEWORK The structure that keeps you organised under pressure

Design questions feel open-ended and panic people. Use this fixed structure and you will sound deliberate even when you are improvising. Do not skip step one — clarifying questions are the clearest signal of seniority.

Step	What you do	What you say
1. Clarify	Establish users, scale, channels, languages, latency target, isolation requirements, budget	"Before I design anything — how many users, which channels, and does this need data isolation between business units?"
2. Requirements	Separate must-have from nice-to-have; define the fallback behaviour	"The critical path is answer-or-route-correctly. Everything else is enhancement."
3. High-level shape	Channels to gateway, to orchestration, to AI service, to data and integrations	"Here's the shape I'd start from, then I'd challenge it against what you've told me."
4. Layer by layer	Walk each layer; name the Azure service and the reason	"For retrieval I'd use Azure AI Search because hybrid search matters here."
5. Cross-cutting	Security, observability, cost, scaling, isolation	"Tenant isolation, telemetry, and per-tenant cost control are what I'd design in from the start, not bolt on."
6. Failure modes	LLM unavailable, retrieval miss, quota exhausted, bad deploy	"If retrieval misses, the bot says so and routes to a ticket — it never improvises."

THE THREE THINGS THAT SEPARATE A DEMO FROM PRODUCTION

Explicit failure paths — every branch has a defined fallback, and the fallback is a useful one.

First-class telemetry — token usage, latency split by retrieval versus generation, fallback rate, escalation rate.

An evaluation loop tied to CI — so a change that regresses quality is caught before users find it.

If you only have time for two things in a design interview, make it the failure paths and the evaluation loop. Almost nobody mentions them, and they are what a senior panel is listening for.

BACKEND QUESTIONS FastAPI, Node, NestJS, and async

Python async versus threading versus multiprocessing a very likely follow-up

Approach	Mechanism	Use it for
asyncio	Single thread, cooperative multitasking; tasks yield at await points	I/O-bound work — LLM API calls, database queries, HTTP integrations. This is the default for an AI service.
Threading	OS-scheduled threads; limited by the GIL for CPU work in Python	Legacy sync libraries you cannot change; light I/O concurrency
Multiprocessing	Separate processes, true parallelism	CPU-bound work — embedding generation locally, large document preprocessing

SPOKEN SCRIPT

"Async is the right answer for roughly ninety percent of what an AI service does, because it's almost entirely waiting on I/O — LLM calls, vector search, ServiceNow REST calls. One event loop handles many concurrent conversations far more efficiently than a thread per request."

The mistake juniors make:

"dropping a synchronous HTTP client into an async route and then wondering why throughput collapsed." If you call a blocking library inside an async handler, you block the entire event loop, and every other in-flight conversation stalls behind it. That's the single most common async bug, and naming it unprompted shows you have actually debugged this in production.

FastAPI — why, and how you harden it

- Native async via Starlette and ASGI, so it handles high-concurrency I/O without a thread per request.
- Pydantic models for request and response validation — validation is free, and it means malformed LLM output fails loudly at the boundary instead of corrupting a downstream system.
- Automatic OpenAPI generation, which matters when you are the API contract owner for a frontend or an integration team.
- Dependency injection for auth, tenant context, and database sessions — clean, testable, and exactly where tenant isolation should be enforced.
- Health endpoints split into liveness and readiness, so a restart doesn't take healthy traffic out of rotation.
- Request timeout middleware, because an unbounded call to an LLM endpoint will eventually hang a worker.
- Structured logging rather than plain text — you cannot correlate a slow response across services otherwise.
- Graceful shutdown so in-flight conversations finish rather than being cut off mid-answer.

Node.js and NestJS — your orchestration layer

- Modular, opinionated structure with dependency injection, which is what makes a large codebase testable and swappable.

- TypeScript end to end — on a project integrating several enterprise systems, catching a shape mismatch at compile time is genuinely valuable.
- **Decorator-driven patterns:** guards for auth and roles, interceptors for logging and response transformation, pipes for DTO validation.
- Feature-based modules — a conversation module, a ServiceNow module, a retrieval client module — so ownership is obvious.

THE HONEST LINE ON NESTJS

NestJS is in your skills, and your orchestration layer at ExxonMobil is Node.js and TypeScript. Be clear about which parts are NestJS and which are plain Node. If you built the orchestration service in NestJS, say so confidently. If it was plain Node with a lighter structure, say that — and then describe how you would structure it in NestJS. Panelists can tell the difference, and honesty here costs you nothing.

DATA AND INFRASTRUCTURE QUESTIONS Your genuine strength — lean in here

MongoDB and Redis — your strongest infrastructure area

Your resume explicitly calls out MongoDB query optimisation and Redis caching at Shell, and the Maximo work gives you real SQL depth. This is where you are strongest and where you should sound most confident.

Concern	MongoDB	Redis
Primary role	Conversation history and document state — flexible schema fits conversation turns	Cache and coordination — not a system of record
Key technique	Indexing and query shape; projection to avoid over-fetching	TTL on everything; eviction policy matched to purpose
Production risk	Unbounded growth and slow scans from missing indexes	Memory exhaustion if treated as a database rather than a cache
Your experience	Optimised MongoDB queries to cut API latency at Shell	Structured Redis caching layers for the same goal

THE LINE THAT SHOWS REAL PRODUCTION EXPERIENCE

"Redis is a cache, not a database — every key gets a TTL, and the system is designed to work correctly when Redis is completely empty. If losing the cache takes down the bot, the design is wrong."

That is a sentence about designing for failure, not about using a technology. It reads as someone who has been paged at three in the morning — which, given your production support experience, you have.

PostgreSQL and relational design your Wipro differentiator

- Know the index types and when each applies — B-tree for equality and range, GIN for full-text and JSON containment, partial indexes for hot subsets.
- EXPLAIN ANALYZE is the tool, not guesswork. Reading a query plan is a core senior skill and almost nobody on a GenAI panel can do it well.
- Connection pooling — an unbounded connection count is one of the most common causes of a database appearing slow under load.
- **The classic relational pitfalls:** N+1 query patterns, missing indexes on foreign keys, and indexes that slow writes without being used for reads.

USE THE WIPRO EXPERIENCE DELIBERATELY

Very few candidates at your level can talk about query plans and index selectivity. When a panel asks a database question, connect it back: "This is the same discipline I applied on the Maximo asset management work at Wipro, where slow queries

over large asset tables were a routine problem. It's the same instinct — find the query doing more work than it needs to." That answer turns a year of early-career work into a differentiating asset rather than something to minimise.

Terraform and CI/CD your delivery strengths

- Module per resource group; environment separation by state or workspace; remote state with locking.
- Plan in pull request, apply on merge, with a manual approval gate for production.
- for_each over count, so resource identity is stable across applies and you don't accumulate orphaned resources.
- Never put secrets in .tfvars — reference Key Vault instead. Terraform state holds secrets in plaintext, which surprises people who have never been paged for it.
- Drift detection on a schedule, so manual console changes surface as a diff rather than as a mystery three weeks later.

ZERO-DOWNTIME DELIVERY — YOU HAVE ACTUALLY SHIPPED THIS

You have genuinely done zero-downtime deployment work, and it is worth stating plainly because it is a real, verifiable production skill that many candidates only claim.

The shape of it: build once, then promote the same artefact through environments rather than rebuilding per environment; run smoke tests against staging; swap traffic only after validation; and keep a fast rollback path. The critical discipline is that the artefact promoted to production is byte-identical to the one tested.

Security — the AI-specific threat surface very likely to be probed

1. **Identity.** Entra ID for users, managed identity for service-to-service calls rather than API keys in code.
2. **Network.** Private endpoints for the model endpoint and databases; VNet integration; nothing exposed publicly that need not be.
3. **Data.** Encryption in transit and at rest, tenant isolation enforced server-side, and PII redacted before anything is sent to a model endpoint — because a prompt is still data leaving your boundary.
4. **Prompt injection.** The genuinely new one. Retrieved documents are data, not instructions, and if any user can write to a knowledge source, that content can carry instructions aimed at the model. Mitigations: explicit delimiters, instruction hierarchy, and output validation before any action is taken.
5. **Tool and action safety.** An allow-list of what the assistant may actually do, scoped by user role, plus human approval for consequential actions. Retrieval is safe; acting is not.
6. **Output guardrails.** Schema validation on anything feeding a downstream system, and a content filter on the model deployment.

THE SENTENCE THAT SHOWS YOU UNDERSTAND THE NEW RISK

"AI systems added a threat surface that didn't exist before. Prompt injection and data exfiltration through carefully crafted queries are LLM-specific, and they need the classic OWASP stack alongside them — not instead of it."

Mentioning prompt injection unprompted, in the context of a retrieved-knowledge system, is a strong senior signal. It shows you have thought adversarially about a system you actually built.

BATCH 4 — BEHAVIOURAL AND CLOSING Q: conflict, failure, mentoring, motivation

Senior panels spend real time here. They are assessing judgement and self-awareness, not just technical skill. Every story below follows the same shape: a real situation, your specific action, and a result you can defend.

Tell me about a time you handled a disagreement the data-over-ego story

The story the old pack used is good. Keep the substance. The strongest version is one where you changed your own mind based on evidence, because that is what proves you are not defensive.

Situation and task

On ITChat, my Python lead and I disagreed about the structure of the AI service. He wanted a single monolithic FastAPI service with the agent logic inline. I wanted to separate the orchestration from the tool-execution layer, because the tool integrations were growing — ServiceNow, then Genesys — and I thought coupling them to the agent loop would make both harder to test and change.

Action

1. I did not argue in the meeting. I built a small spike implementing both approaches against one representative use case.
2. **I measured the three things we would actually disagree about:** how long it took to add a new tool, how easy it was to debug a failed conversation, and what test coverage we could realistically achieve.
3. I brought the data to the next design review and presented both options honestly, including the costs of my own proposal.
4. Then I explicitly invited him to argue against my design, because I knew he had a point I was underweighting.

Result

The team adopted the split — but with a change I had not proposed, which is the part worth telling. We ended up using a framework's native state-machine support instead of the hand-rolled orchestration layer I had been proposing. It was better than either of our original positions, and I would not have got there if I'd won the argument on my own terms.

WHY THIS LANDS WITH A SENIOR PANEL

You resolved it with a spike and data rather than authority — which is how you will have to resolve it as a senior engineer, since you will not have positional power over your peers.

You ended up adopting someone else's better idea. That is a strong, credible signal of low ego, and it is much rarer than candidates admit.

It demonstrates the actual skill being hired for: influencing technical direction without being the decision maker.

Tell me about a failure use the Sam intent regression

Your resume shows you shipped Azure CLU intent models trained on customer utterances at Shell. A failure story around a conversational AI change is far more credible than a generic one, and it demonstrates you understand something genuinely important about ML systems.

Situation

Early on the Sam project, I made what I thought was a small change to the conversational language understanding configuration — adding an intent and training utterances for it. I was reasonably confident the new intent was distinct enough from the existing ones.

Task

The consequence was that I had no systematic way to know whether the change was actually safe. We didn't have a regression check on the conversational models, so my confidence was the only thing standing between that change and production.

Action

1. I caught it in the fallback rate shortly after deployment, rolled it back, and told my lead directly that I'd skipped a validation step rather than letting it look like a mystery.
2. **I wrote it up honestly:** root cause was missing process, not a technical subtlety nobody could have known.
3. I then built a regression harness — every conversational model change runs a confusion-matrix evaluation against a frozen test set before it can merge.
4. I made it a CI gate rather than a recommended practice, because a recommendation is a thing people skip when they're behind.

Result

That approach held. The real lesson was about how ML systems differ from conventional software: failure modes here are statistical rather than deterministic, so “there are no small changes— a change that looks isolated can quietly cannibalise neighbouring intents. That reframing is what changed how I approached every model change afterwards, and it's a point I now raise whenever a team treats a conversational change as a low-risk edit.

THE INSIGHT IS THE ANSWER

Anyone can say “I made a mistake”. What a senior interviewer is listening for is whether you extracted a transferable principle. — There are no small changes in ML systems, because the failures are statistical rather than deterministic. — is a genuinely good insight, and it is true.

Tell me about mentoring or knowledge transfer [lead with the handover](#)

SPOKEN SCRIPT

The most concrete example right now is the knowledge transfer on ITChat as part of my notice period. The incoming developer is strong in Python but new to LangChain-based RAG and to how our retrieval pipeline is put together.

My approach was structured rather than ad-hoc.

One focused session per architectural layer rather than a continuous stream of explanations, because information delivered while someone is also trying to absorb it doesn't stick. Then, for each layer, a small concrete change to make — comprehension comes from doing, not from watching.

I also wrote up a runbook from past production incidents: what failed, why, and how we resolved it. And I recorded the reasoning behind non-obvious design choices, because those are the things that get lost in a handover and then get reinvented badly a year later.

On the coaching side, I deliberately handed over progressively rather than doing the work myself. The goal was that he'd own the system independently, not that the handover looked complete while I still held all the context.

IF YOU ARE ASKED ABOUT MENTORING MORE BROADLY

Have one more example ready — a junior you onboarded, or a colleague you helped with a specific technical area. The runbook-from-incidents habit is a good concrete detail to expand on, because it shows you build systems for other people, not just for the immediate task.

Panels at senior level are also assessing whether you raise the level of people around you. Any answer that is only about your own output will read as a mid-level answer.

Tell me about a time you disagreed with your manager the fine-tuning versus RAG story

The reasoning in this story is sound and worth keeping. Use it, but soften the ending — do not imply your manager was wrong. You want to show healthy disagreement, not a conflict narrative.

Situation and action

A manager wanted to fine-tune a model for a specific use case, on the reasoning that it would feel more enterprise-native. I thought RAG would hit the accuracy bar faster and far more cheaply. I didn't push back in the meeting. I took a couple of days and built the comparison: fine-tuning cost and maintenance overhead, expected accuracy uplift against RAG, and time to market.

Result

I brought it as a one-to-one, framed as — I want to make sure we're picking the right path, here's what I dug into, and here's where I'm confident and where I'm not. He pushed back on parts of my reasoning, and he was right to. We went with a hybrid, using RAG for the knowledge-heavy paths. The more useful outcome was that we'd established a precedent: when the team said fine-tuning, the expectation became a comparison rather than a decision already made.

THE FRAMING TO AVOID

Never tell a story where your manager was simply wrong and you were right. Even if that is the truth, it reads as ego, and panels are testing exactly that.

The winning version is one where you changed the decision process rather than winning a single argument. — The expectation became a comparison rather than a decision already made.— is the senior-level outcome.

What are your strengths? three, each backed with evidence

1. **Backend plus AI in one person.** Most candidates are strong on one side and shaky on the other. I can design the orchestration service, build the AI service, and own the Azure deployment as one coherent system — which is what made ITChat ship rather than stall at the integration stage.
2. **Production discipline.** I default to monitoring, evaluation, and runbooks. Caching, query tuning, async processing, zero-downtime release — the unglamorous work that decides whether a system survives contact with real traffic.
3. **Calibrated honesty.** If I don't know something I say so, and then I say how I'd approach it. I held that line on the AI-900 depth versus my production experience, and I'd do the same with your tooling.

What are your weaknesses? two, both real, both with mitigation

1. **Formal theoretical depth.** I'm strong on applied systems and production delivery. If you asked me to derive a transformer architecture from scratch on a whiteboard, I'd struggle. I've been working through the material deliberately to close that, but I'd be honest that I'm currently trading some theoretical depth for production impact — knowingly.
2. **Delegation.** I get pulled into fixing things myself when the better instinct is to hand over and coach. I'm actively working on it — the ITChat handover is deliberately a practice in handing over rather than doing, including resisting the urge to take the keyboard in code reviews. It's a real shift and I'm not all the way there.

WHY THESE TWO WORK

Both are real, neither is disqualifying, and both have visible mitigation in progress. That combination is exactly what a senior panel wants.

Avoid the two classic traps: a fake weakness that is really a strength in disguise (— I work too hard—), and — I'm a perfectionist—, which everyone has heard a thousand times this week.

Why do you want to join us specifically? three concrete reasons

- 1. Scope fit.** The JD describes multi-tenant chatbot deployment, RAG integration, enterprise system integrations, omni-channel rollout, and bot performance metrics. That is almost a one-to-one match with what I've been doing at ExxonMobil for the last twenty-five months. I'd be productive in weeks, not quarters.
- 2. Enterprise complexity.** A group spanning multiple business companies has genuine multi-tenancy needs and real production traffic. That's the environment where conversational AI architecture decisions actually matter, as opposed to being a demo.
- 3. The blue-collar use case.** Supporting a workforce that includes Tamil-speaking shop-floor workers is a more interesting technical problem than a typical English-only enterprise chatbot. Voice, regional language, and low-bandwidth channels are real constraints that produce real design decisions — and it's a problem I've already thought about.

THE LINE TO USE IF THEY PRESS ON RELOCATION

"I'm based in Faridabad in Delhi NCR and I'm open to relocating. I'd want to understand the joining timeline, and I'm happy to plan the handover around it." Then stop. Do not over-explain, and do not apologise for being based in the NCR.

Do you have any questions for us? always ask 3—5

Asking nothing is a senior-level red flag. Pick three or four from these. They should sound like you are already working there, not like you are evaluating from the outside.

Theme	Question	Why it lands
AI direction	How is the AI roadmap structured here — is there a central platform team, or does each business unit own its own bot stack?	Shows you understand the org-level question behind the tool question
Evaluation	What does the evaluation loop look like for the production bots today — is there a golden set, and how do you catch a regression before a user does?	Signals you care about quality, which is the thing most teams skip
Current state	What are the current accuracy and fallback rate baselines for the bots in production?	Asking for baselines is a senior move — it implies you think in terms of measurable outcomes
Team	Who would I be working with day to day, and how is the team split between bot authoring, backend, and platform work?	Practical, and it tells you whether the role is what you expect
Priorities	How is priority split between new bot development and improving bots already in production?	Shows you understand the real tension in any enterprise AI team
Success	What does success in this role look like at six and at twelve months?	Signals that you are already thinking about the job rather than the interview
Process	How do production releases happen, and is there a change-management or approval process specific to AI changes?	Connects to your ServiceNow and enterprise change-management experience
Logistics	What's the day-to-day work mode for this role, and where is the team based?	Gets the location question on the table before it becomes an objection

THE TWO BEST QUESTIONS TO LEAD WITH

The evaluation question and the multi-tenant roadmap question. Both are the questions a person who has actually run a production RAG system would ask, and both steer the conversation toward your strengths.

End with something forward-looking rather than logistical. Closing on — what does success look like at six months — leaves a final, positive impression.

Salary expectations HR round — fill in your real numbers

PREPARE THIS BEFORE THE CALL

Know your current all-inclusive CTC, your target range, and your floor. Write them down. Being asked for a number you have not thought about is where leverage is lost.

Always give a range, never a single number. Always refer to total CTC, not base alone — base, variable, joining bonus, and retention all count, and a range on total package gives you room.

Tie the number to the scope of the role, not to market data. — Given the seniority and the GenAI architecture scope, I'm looking in the range of X to Y, flexible based on the total package and the growth trajectory.—

Never lie about your current CTC. Large firms verify, and it is not worth the risk.

FINAL — SCORECARD, GAPS, AND CHECKLIST

An honest self-assessment, recalculated against your actual resume rather than the old generic pack. Use it to decide where to spend your remaining preparation time.

Honest scorecard your starting position

Dimension	Score	Assessment
GenAI and RAG production depth	8.5 / 10	Five years of it, on live enterprise systems with real integrations. Not a 9 because your evidence is strong on RAG specifically but thinner on formal evaluation tooling and shipped agent frameworks.
Conversational AI and channels	9 / 10	Your strongest area. Bot Framework, Azure CLU, four channels, Adaptive Cards, multi-turn, fallback, escalation. Rare, and under-priced in your own self-assessment.
Backend engineering	8 / 10	FastAPI, Node, TypeScript, microservices, async, conversation state. Solid and production-proven.
Enterprise integrations	9 / 10	ServiceNow across three modules plus Genesys Live Chat, on enterprise client systems. Very strong, and directly relevant to this JD.
Databases and performance	8 / 10	MongoDB optimisation, Redis caching, SQL from Maximo, plus explicit async performance work. The Wipro year adds real depth.
Azure and delivery	7.5 / 10	App Service, Functions, DevOps, GitHub Actions, Terraform are real. Weaker on the security and networking layer — private endpoints, VNet and Key Vault are design knowledge, not shipped.
System design	7 / 10	Good instincts, but rehearse the multi-tenant answer until it is automatic. The content is there; delivery under pressure is the risk.
Power Platform and Co-Pilot Studio	3 / 10	The real gap. Manageable with the bridge framing, but do not let it go unaddressed.
Theoretical ML depth	5.5 / 10	Weakest technical area. If pushed on attention maths or derivation, you will struggle. Volunteer the gap before you are asked.
Certifications	8.5 / 10	Four relevant certs including two Claude and AI-900. Lead with these — the old pack's "no certifications" framing is now wrong.
Communication and presence	8 / 10	Good, but the risk is over-explaining when nervous. Practise brevity.

Where you are stronger than you think correct your self-assessment

- Your conversational AI depth is genuinely strong, and this is a conversational AI role. Four channels, five years, intent models trained on real utterances — most candidates at your level have one channel and no NLU work at all.
- Your enterprise integration breadth across three ServiceNow modules is named directly in the JD and is not common.
- You have two employers and three distinct enterprise systems behind you, which reads as range rather than repetition.
- Your certifications are considerably better than the old pack assumed. Use them, and lead with them.
- The Wipro year gives you database and production fundamentals that most AI-only candidates completely lack.

Gaps to prepare for — ranked by likelihood of being probed

Gap	Likelihood	How to handle it
Co-Pilot Studio	Very high	Use the bridge framing from Batch 1. Address it before they raise it. Never bluff.
Location — Delhi NCR versus Chennai	Very high	State willingness to relocate early and calmly. Have your notice period in weeks ready.

Gap	Likelihood	How to handle it
Theoretical ML depth	Medium	Volunteer it in the weaknesses answer so it lands as self-awareness rather than an ambush.
True multi-tenant platform experience	Medium-high	Be honest: isolated client deployments, and tenant isolation designed for. Then walk through the bridge architecture.
Power Platform breadth — Power Automate, Dataverse, Power BI	Medium	Name the concepts you know, be clear you have not shipped them, and ask how much of the stack is in scope.
Codable depth — DSA or live coding rounds	Low-medium	If the process includes a coding round, that is a different preparation track. Confirm the process early and prioritise accordingly.

Top twelve questions most likely in your actual interview revised for this JD

1. Walk me through your RAG pipeline end to end.
2. How do you handle hallucinations in production?
3. How do you improve the accuracy of the assistant's responses?
4. Explain your chunking strategy, including for PDFs with tables and images.
5. Async versus multithreading in Python — when do you use which?
6. Design a multi-tenant chatbot architecture for a group with multiple business units.
7. Have you worked with Co-Pilot Studio specifically?
8. What are your Azure services, and why those?
9. How do you secure an Azure OpenAI deployment?
10. Tell me about a production incident you resolved.
11. Why are you leaving TCS, and why this role?
12. What is your notice period and earliest joining date?

Panel simulation — how this interview most likely goes

MOST LIKELY OUTCOME: HIRE, WITH A NOTE ON THE CO-PILOT STUDIO RAMP

A panel will likely see strong alignment on RAG, conversational AI, ServiceNow integration, and multi-channel deployment, with one clear gap on the specific Microsoft authoring tool. That is a routine hire-with-onboarding-plan outcome, not a rejection.

The realistic recommendation is an offer at senior level, with a two-to-four week Co-Pilot Studio ramp built into the first quarter and an explicit note that the panel should pair you with someone strong on Power Platform tooling in the interim.

Two things could change that outcome, and both are within your control. First, if you are vague about Co-Pilot Studio instead of honest, the gap becomes a competence question rather than a tooling question. Second, if relocation is not addressed proactively, it can become a logistics veto regardless of technical fit.

Pre-interview checklist

Two to three days before

- Rehearse the 75-second introduction and the 90-second ITChat description out loud, three times each, timed.
- Rehearse the three STAR stories until you can deliver each in ninety seconds with no notes: the RAG accuracy problem, the architectural disagreement, and the Sam intent regression.
- Walk the multi-tenant architecture out loud with pen and paper. Do it twice.
- **Write down your real numbers:** current CTC, target range, floor, notice period in weeks, earliest joining month.
- Research TVS Group the night before — recent announcements, their digital transformation direction, and anything on AI or workforce technology.

- Re-read the JD once more and note the three requirements you have the best evidence for.

The day before — fill these in, do not skip them

Item	Your answer — write it here
Notice period (duration + release month)	
Earliest joining date	
Other processes and their stage	
Current all-inclusive CTC	
Target range and absolute floor	
Relocation — firm or flexible	
Who will provide a reference, and when	

Thirty minutes before

- Test camera, microphone, and internet. If it is an online round, dial into the meeting early rather than at the time.
- Have the JD open on screen, plus your resume and this pack's verified profile table.
- Water within reach.
- Read your three STAR story headlines once more — situation, conflict, insight — not the full text.
- One long slow breath. You have done the preparation; now you just have to talk about work you have actually done, which is the easiest kind of talking.

In the room — five reminders

1. Answer the question that was asked. Do not perform your prepared answer at any cost.
2. Be specific. Name the service, the pattern, the number, the reason.
3. If you do not know, say so, then say how you would approach it. This is a strength, not a failure.
4. Every technical answer ends at an outcome — accuracy, latency, deflection, cost, ticket volume.
5. Speak at roughly two-thirds your natural speed. Nervous speakers accelerate, and clarity is what you are being scored on.

One-line summary to carry into the room

Five years building production conversational AI for Fortune 500 clients — RAG pipelines that work, integrated with the systems businesses actually run on, deployed to real users. I don't know Co-Pilot Studio yet; I know every pattern underneath it.

You have shipped enterprise AI systems across two employers, three client systems, and four channels. Most candidates at your level are presenting a project. You are presenting a career.

TOP TWELVE QUESTIONS — WITH MODEL ANSWERS [Rehearse these](#)

These are the questions most likely to come up in your interview, each with a model answer written to your actual experience. This section is the backbone of your technical round.

DO NOT MEMORISE THESE WORD FOR WORD

Memorise the structure of each answer — the layers, the order, the conclusion — then say it in your own words. A word-for-word answer sounds rehearsed, and panels detect that quickly.

Rehearse Q1, Q6 and Q10 most: those three you will almost certainly be asked, and they are where structure matters most. Time each between ninety seconds and two minutes. Q2, Q3 and Q8 should come out under a minute.

Q1. Walk me through your RAG pipeline end to end [memorise — your flagship answer](#)

SPOKEN SCRIPT

The sources are the enterprise knowledge that IT support is genuinely asked about: policy documents, IT runbooks, and knowledge base articles.

The pipeline has five stages.

Ingestion and chunking first. That's where the first real accuracy decision happens, because how you break a document determines what can be retrieved later. I preserve section headers as metadata, because that structure is what retrieval keys on.

Then embeddings, and critically the refresh cycle. When a policy document changes, its embeddings have to be regenerated. If you don't, the system confidently retrieves outdated guidance — which is a quiet failure mode, because nothing errors.

At query time it's retrieval. Honestly, the majority of quality improvement in this system came from retrieval work, not prompt changes or model changes. If retrieval returns the wrong chunk, no prompt can rescue the answer.

Then generation — prompt engineering and grounding, so the model answers from retrieved context rather than from memory. And a defined fallback: when the knowledge base doesn't cover the question, we say so and route to a ticket or a human, rather than guessing.

Over the top of all of that sits conversation state in the orchestration layer, so multi-turn follow-ups don't each start from zero.

Q2. How do you handle hallucinations in production? [four-layer answer](#)

SPOKEN SCRIPT

I treat hallucination as a systems problem rather than a model problem, and I attack it at four layers.

One, prevention.

Ground every answer in retrieved context, instruct the model to answer only from that context, keep temperature low on factual paths, and give it an explicit sanctioned way to say it doesn't know. That last part matters more than people expect — a model that has a graceful 'I don't know' path will actually use it.

Two, detection.

Check that claims map back to retrieved content. Where the stakes justify it, a second LLM pass acts as a judge for faithfulness. That's expensive, so it goes on the high-stakes paths, not every response.

Three, mitigation at the boundary.

A refusal path that's actually useful — 'I don't have that in my knowledge base, shall I raise a ticket?' — rather than a

dead end. And escalation to a human through the channel you already have, which at Shell was Genesys Live Chat.

Four, monitoring.

Fallback rate and escalation rate as first-class metrics, with sampled review. A rising fallback rate is an early warning that something upstream has degraded — usually retrieval.

Q3. How do you improve the accuracy of the assistant's responses? [rank the levers](#)

SPOKEN SCRIPT

The honest answer is that it's a stack, and the levers are not equally valuable. Most teams jump to prompt engineering. I'd fix retrieval first, because it gives a much better return.

In order of impact for my use case:

Retrieval quality, first. Chunking that respects document structure, hybrid keyword-plus-vector search so exact terms like policy numbers and error codes match as reliably as conceptual questions, a reranking step, and metadata filtering to scope results to the right document set.

Then query handling.

Rewriting ambiguous questions and decomposing multi-part questions before retrieving.

Then grounding and prompts.

Answer-only-from-context, few-shot examples for format consistency, an explicit refusal instruction.

Then model choice and parameters.

Route simple intents to a smaller model and keep temperature low on factual paths.

And underneath everything, an evaluation loop.

A golden set of real support questions, regression-checked before anything ships. Without that, none of the above is measurable — and you have no idea whether a change helped or quietly made things worse.

Q4. Explain your chunking strategy, including PDFs with tables and images [your strongest technical area](#)

SPOKEN SCRIPT

Chunking is content-type specific, and I'd argue there's no universal chunk size — the right unit depends on the document type and the question being asked.

The way I think about it:

Narrative documents like policies and SOPs — recursive character splitting, roughly 500 to 1000 tokens with overlap, because overlap stops a sentence being split across two chunks and losing its meaning.

Structured documents with clear headings — split on section boundaries and keep the heading as metadata, because the heading is often the strongest retrieval signal you have.

Tables and tabular data — extract as a unit and never split a row. A table split across chunks is unusable, and that's one of the most visible and most fixable RAG failures.

Runbooks and step procedures — keep each step with its parent step. An orphaned instruction is dangerous, not just inaccurate.

Scanned PDFs and images — OCR first, then chunk the extracted text. With no text layer, nothing is retrievable until OCR runs.

And two things across all of them:

Strip headers, footers and page numbers before chunking, because they pollute every embedding. And always attach metadata — source, section, last-modified, and access scope. That last one matters most in multi-tenant, because it's how you stop one business unit retrieving another's documents.

Q5. Async versus multithreading in Python — when do you use which? very likely follow-up

SPOKEN SCRIPT

Async is the right answer for roughly ninety percent of what an AI service does, because it's almost entirely waiting on I/O — LLM calls, vector search, ServiceNow REST calls.

Async is cooperative: a single thread where tasks voluntarily yield at await points. One event loop handles many concurrent conversations far more efficiently than a thread per request. Threading is preemptive and OS-scheduled, and in Python the GIL limits how much genuine parallelism you get for CPU-bound work anyway. Multiprocessing gives real parallelism across processes, and it's correct only for genuinely CPU-bound work — local embedding generation, or heavy document preprocessing.

The mistake juniors make

is dropping a synchronous HTTP client into an async route and then wondering why throughput collapsed. If you call a blocking library inside an async handler you block the entire event loop, and every other in-flight conversation stalls behind it. That's the most common async bug, and naming it unprompted shows you've actually debugged it.

Q6. Design a multi-tenant chatbot architecture for a group with multiple business units TVS scenario — near-certain question

SPOKEN SCRIPT

Before I design anything — how many business units, roughly what scale, which channels, and does each unit need isolated data or just isolated configuration? Those answers change the design significantly.

At a high level, three models.

Silo — separate infrastructure per tenant: strongest isolation, highest cost, correct only for regulated or very large tenants. Pool — everything shared with a tenant ID per row: cheapest, weakest isolation. And bridge — shared compute and services with isolated data. That's almost always right for a group with multiple business companies.

Walking through the bridge model:

Shared compute — one orchestration service and one AI service serving all tenants, with isolation enforced in the data layer rather than by spinning up infrastructure per business unit.

Tenant identity derived server-side. This is the golden rule: tenant context comes from the validated identity token, never from a header or parameter the caller supplies. If a client can name its own tenant, the whole model collapses.

Storage isolation — a separate index or container per tenant is cleanest and easiest to audit; a shared store with a mandatory tenant filter is the cost-efficient option. For retrieval specifically, embeddings carry tenant metadata and every query filters on the authenticated tenant.

Per-tenant cost control — token budgets, throttling, separate indexing schedules, so one tenant's ingestion job never starves another's traffic. And model routing, so simple intents go to a smaller model. That's where cost actually lives in an LLM system.

And I'd design the failure paths in from the start

— a retrieval miss means say-so-and-route-to-a-ticket, never improvise; quota exhaustion means shed by tenant priority rather than failing everything.

Q7. Have you worked with Co-Pilot Studio specifically? honest + bridge framing

SPOKEN SCRIPT

No — not the tool itself. My conversational AI work has been code-first: Microsoft Bot Framework on Node.js, FastAPI

on Python, LangChain for the RAG layer, deployed across Teams, Zoom, Google Chat and web.

That said, the underlying concepts map closely. Topics and triggers are the equivalent of the intent routing and multi-turn flows I've built — and Azure CLU solves the same intent-and-entity problem that topics solve. Custom connectors are the REST integrations I built against ServiceNow and Genesys. And grounding over enterprise knowledge is exactly the RAG pipeline I run at ExxonMobil today.

So I'd expect a short ramp. I'd want to understand the authoring model properly — topics and triggers, knowledge sources, connectors, and how generative orchestration plans multi-step actions — rather than assume I already know it. The underlying concepts map closely, so I don't think the ramp is long; but I haven't used the tool, and I'd rather learn it properly than pretend otherwise.

Q8. What are your Azure services, and why those? be precise about shipped vs designed

SPOKEN SCRIPT

What I've actually shipped: Azure App Service and Azure Functions hosting the Node.js orchestration layer and the Python AI layer, Azure DevOps and GitHub Actions for CI/CD across environments, Terraform for infrastructure as code, and Application Insights for monitoring.

The reasoning is fairly unglamorous. App Service and Functions because the workload is request-response plus scheduled async work — ingestion and reindexing — and I didn't need the control or the cost of containers. Terraform and a real pipeline because you cannot hand-manage multi-environment Azure and still claim zero-downtime delivery. We promote the same artefact through environments rather than rebuilding per environment, which is the only way the thing in production is genuinely the thing you tested.

On the AI side, Azure OpenAI for generation and embeddings. For an enterprise client, Azure is realistically the only option, for four reasons: data residency, private networking, Entra ID and managed identity instead of API keys in code, and a formal SLA.

Where I'd be careful in my own answer:

I'd describe private endpoints, VNet integration, Key Vault and managed identity as design knowledge rather than claiming them as shipped, because they are. That honest split is respected far more than an inflated list.

Q9. How do you secure an Azure OpenAI deployment? the AI-specific angle is the differentiator

SPOKEN SCRIPT

Defence in depth — no single control is enough, and an answer that's only about the API key is a junior answer.

Network:

a private endpoint on the Azure OpenAI resource so it isn't reachable from the public internet, VNet integration for App Service and Functions, and NSG rules restricting egress.

Identity:

managed identity for service-to-service calls rather than API keys in code, Entra ID RBAC with least privilege, and keys in Key Vault with rotation if you use them at all. The goal is zero long-lived credentials.

Data:

encryption at rest, data residency locked to a region, a contractual no-training guarantee, and PII redaction before anything reaches the model — because a prompt is still data leaving your boundary.

And the one that's new because of AI:

prompt injection. Retrieved documents are data, not instructions. If any user can write to a knowledge source, that content can carry instructions aimed at the model. So you wrap context in explicit delimiters, treat retrieved content as data rather than instructions, and validate output before taking any action. Retrieval is safe; acting is not — so tool

access gets an allow-list scoped by user role, with human approval for consequential actions.

Q10. Tell me about a production incident you resolved your technical STAR story

SPOKEN SCRIPT

On ITChat, after launch the assistant's usefulness plateaued. It was deployed, integrated, working in Teams — but users weren't trusting it for policy questions, and a meaningful share of conversations ended in a fallback or a ticket. Simple requests deflected fine; the questions that mattered to users were the ones failing.

I owned working out whether it was retrieval, generation, or the source data — and I was deliberately not going to just tune the prompt until the demo looked good.

What I did:

I pulled a sample of real conversations and classified the failures. Were we retrieving the wrong chunk, retrieving nothing, or retrieving correctly and answering badly? In most cases the correct document was in the knowledge base and never came back — so prompt tuning would have been wasted effort.

The root cause was chunking. Policy and runbook documents were split in ways that broke meaning: tables split across chunks, and section headings separated from the text they described, so a retrieved chunk had no usable context.

The fix was to the pipeline, not the prompt.

Reworked chunking to respect document structure and keep section metadata attached. Added hybrid keyword-plus-vector retrieval so exact terms like policy numbers matched as reliably as conceptual questions. Added a reranking step. And tightened the fallback so the bot says so and offers a ticket rather than improvising something plausible.

Then I put a golden set of real support questions in as a regression check, so a future chunking or prompt change couldn't quietly undo the improvement.

Accuracy on the golden set improved materially, and fallback rate on policy questions dropped noticeably. The more valuable outcome was that quality stopped being a matter of opinion — we could tell whether a change actually helped.

Q11. Why are you leaving TCS, and why this role? pull toward growth, never push away

SPOKEN SCRIPT

Two reasons, and they're related.

First, focus. For the last five years conversational AI has been the core of my work rather than a side capability. I want the next role to go deeper on multi-tenant bot architecture, RAG at scale, and evaluation frameworks that actually tell you whether quality improved. I'm at the stage where I want to own that end to end, and to be building on a platform rather than building the platform.

Second, ownership. I want to be closer to outcomes, with sharper feedback loops and a real hand in architectural decisions earlier in the lifecycle.

And why this role specifically:

The JD describes multi-tenant deployment, RAG integration, enterprise system integrations, omni-channel rollout, and bot performance metrics. That's almost a one-to-one match with what I've been doing at ExxonMobil for twenty-five months, so I'd be productive in weeks rather than quarters. The group also has genuine multi-tenancy needs across multiple business companies, and supporting a workforce that includes Tamil-speaking shop-floor workers is a more interesting technical problem than a typical English-only enterprise chatbot — voice, regional language and low-bandwidth channels produce real design decisions.

Q12. What is your notice period and earliest joining date? FILL IN YOUR REAL DATES

SPOKEN SCRIPT

My official notice is [N] days, and my expected last working day is [date]. So realistically I'd be available to start in [month].

If your timeline needs me sooner, I'm open to discussing early release — I have an active handover in progress, so it's a conversation rather than a problem. I'm also completely open to relocating; I'm currently in Faridabad in Delhi NCR and that isn't a constraint.

FILL THIS IN BEFORE THE INTERVIEW

This is the one answer you must not rehearse vaguely. Know your notice period in days, your actual last working date, and the month you'd realistically start — then say all three, in that order.

Adding the relocation sentence at the end is deliberate. It gets the location question answered before it becomes an objection, and it frames you as someone planning the move rather than hesitating about it.

TRENDING 2026 QUESTIONS The current edge of your field

These are questions panels are asking in 2026 that were not standard two years ago. Most of your competition will not have prepared for them, so a solid answer here is a genuine differentiator. Each is written so you can answer it honestly using what you actually have.

T1. What is MCP, and have you used it? know the concept, claim nothing more

MCP — the Model Context Protocol — is a real topic in 2026 and you may be asked about it. You have not built with it, so the correct position is: you understand the concept and why it matters, and you are clear that you have not shipped it. That is a perfectly respectable answer and it is far safer than bluffing.

- **The concept, so you can speak accurately:** MCP is an open protocol for how an application exposes tools and resources to a language model, so clients can consume them without writing a bespoke integration for every system.
- **Why it matters:** It removes per-integration cost — the server publishes its capabilities once, and compatible clients pick them up with the schema already described. It also standardises how an agent discovers what it can do.
- **Where it is headed:** Support is appearing across agent platforms, including the Microsoft ecosystem. For a multi-tenant enterprise deployment, the interesting question is governance — who is accountable when a third-party server exposes something the agent should not call, and how tool access is allow-listed and audited.
- **Your position:** You have read about it and understand why an enterprise would standardise tool integration this way. You have not built an MCP server or integrated one, and you should say so plainly.

SPOKEN SCRIPT

If asked directly:

"I've read about MCP and I understand why it matters — it standardises how an application exposes tools to a model, so you don't write a bespoke integration per system. I haven't built one, so I can't speak from production experience with it. But architecturally it's the direction I'd expect things to go in, and the part I'd be most careful about in an enterprise setting is governance: who's responsible for what a third-party server exposes, and how you allow-list and audit tool calls."

HOW TO HANDLE THIS SAFELY

Say "I've read about it" or "I understand the concept" — never "I've worked with MCP" or "we used MCP". The moment you claim hands-on experience you cannot defend under follow-up questions, the interviewer's trust in everything else you have said drops sharply.

If they want to test you on MCP specifics, it is fine to say: "I haven't used it in production, but if you want to talk through how I'd design around it, here's how I'd think about it." Offering reasoning is not the same as claiming experience, and the panel knows the difference.

T2. Workflows versus agents — when would you use each? a reasoning question, not trivia

This is the sharpest conceptual question in agentic AI right now. The good answer is about restraint, not enthusiasm — panels ask it to see whether you reach for agents by default.

	Workflow	Agent
How it runs	LLM and tools orchestrated through predefined code paths	The LLM dynamically directs its own process and tool usage
Predictability	High — you know the sequence in advance	Lower — behaviour emerges from the model's decisions

	Workflow	Agent
Best for	Well-defined tasks with known steps	Open-ended tasks where the path cannot be fully specified
Cost and latency	Lower — bounded number of calls	Higher — variable, and can loop
Debuggability	Straightforward	Harder — needs tracing and bounded iteration

SPOKEN SCRIPT

"My default would be a workflow. The most successful systems I've seen use the simplest thing that works — and for an IT support assistant, most questions are a predictable sequence: classify intent, retrieve, then answer or route. That's a workflow, and it should be a workflow, because it's predictable, cheap and debuggable."

An agent earns its place where the path genuinely can't be predetermined — a diagnostic request where you don't know in advance which two or three things to check, or a case crossing several systems where you need to decide what to try next. Even there I'd bound it: a maximum iteration count, explicit state so you can see where it is, and defined escalation when it runs out of road.

The trap

is reaching for an agent because it sounds more advanced. An agent trades latency, cost and predictability for flexibility on tasks that may not need it. If a workflow gets you there, the workflow is the better engineering choice."

WHY THIS ANSWER SCORES WELL

It demonstrates the most valuable instinct in production GenAI: knowing when NOT to use the more sophisticated tool. Most candidates answer with enthusiasm for agents, which is exactly the wrong signal for a senior role.

It also proves you have shipped a bounded, predictable system — which is what your actual work was.

T3. What is context engineering, and how does it differ from prompt engineering? emerging term — use it

SPOKEN SCRIPT

"Context engineering is the broader discipline. Prompt engineering is writing a single instruction well. Context engineering is deciding what the model should see at all — what to retrieve, how much of it, in what order, what to compress, what to leave out — and then structuring that so the instruction sits inside it effectively."

It exists because context is a finite budget with a real cost. More context isn't automatically better: it costs tokens, it dilutes attention, and it raises the chance the model latches onto something irrelevant. So the work becomes selection and ordering rather than writing.

For me it's mostly a retrieval problem in practice:

getting the right four chunks rather than the right forty. Reranking, compression, and dropping retrieved material that doesn't actually support the answer. I'd expect this to keep absorbing what used to be prompt-level work."

Use this term deliberately. It is current, it is genuinely in use, and saying it accurately signals that you read beyond prompt guides.

T4. When would you use a small language model instead of a large one? cost engineering question

SPOKEN SCRIPT

"The case for a smaller model is that most enterprise traffic is not hard. In an IT support assistant, the large share of questions are intent classification, entity extraction, simple lookups, ticket summarisation and routing. Those don't need frontier capability, and running them on a large model is paying premium price for a result nobody can distinguish."

So I'd route by task. A small, fast model for classification, extraction and short structured answers; the larger model

only when the task genuinely needs reasoning over long context. And keep temperature low on both.

The part people skip:

a small model still needs grounding and the same fallback discipline. Swapping to a small model doesn't remove the hallucination risk, it just changes the shape of it — a small model will be confidently wrong in different ways than a large one. And the routing layer becomes production-critical, because a misrouted request silently gets the wrong model."

T5. How do you control GenAI cost at enterprise scale? finance will ask this

- Route by task and by tenant — the largest single lever. Most traffic does not need the most expensive model.
- Token budgets per conversation and per tenant, so one tenant cannot consume the allocation.
- Prompt caching for the stable parts of the system prompt and for repeatedly retrieved context.
- Retrieval quality as a cost lever — retrieving fewer, better chunks cuts both token cost and latency.
- Cache responses for genuinely repeated questions, with care around personal or access-scoped data.
- Track cost per conversation and per business unit, not just aggregate spend. You cannot optimise what you do not attribute.

SPOKEN SCRIPT

"The biggest lever isn't prompt optimisation, it's model routing. If seventy percent of traffic is classification and short answers, running that on a large model is the single most expensive mistake available."

Behind that: token budgets per tenant so consumption is attributable, caching for the stable parts of the prompt and for repeated retrievals, and retrieval quality — because fewer, better chunks is simultaneously better for accuracy and cheaper.

And measurement:

cost per conversation and per business unit, not aggregate. Once each business unit can see its own number, behaviour changes — and you can have an informed conversation about which use cases deserve which model."

T6. How do you evaluate an LLM system beyond vibes? your strongest area — lean in

SPOKEN SCRIPT

"I split it into offline and online, because they answer different questions."

Offline

— a golden set of real questions with known-good answers, scored on faithfulness (is the answer supported by the retrieved context), answer relevance, and context precision and recall. Ragas computes these; DeepEval lets you write them as tests that run in CI. For me the golden set matters more than the tooling, because it's built from real support questions rather than synthetic ones.

Online

— fallback rate, escalation rate, thumbs up or down, latency split between retrieval and generation, and token cost. These catch what a golden set can't, because real users ask things you didn't anticipate.

The loop

is that sampled production failures get added to the golden set. That's how the set stays honest instead of drifting away from what users actually do.

And the discipline that makes it stick:

prompts and retrieval config are versioned code with regression tests. Without that, you can't tell whether a change helped or made things worse — and most teams genuinely can't."

YOUR HONEST POSITION ON TOOLING

If pressed on how automated your eval stack is, be straightforward: you've run offline evaluation and golden-set regression work in practice, and you know the established frameworks, but your day-to-day has been domain-specific golden sets plus production metrics rather than a fully automated platform. That is a very normal and credible senior position — and it is a fair thing to strengthen.

QUESTIONS THEY SHOULD BE ASKING YOU Use these to steer the interview

These are the questions I would ask if I were sitting on this panel. Bringing two or three of them up yourself does something most candidates never do: it demonstrates that you already understand the job's real problems, and it converts the interview into a conversation between two people who know the domain.

Ask these if the conversation is going well two or three is plenty

- **"What's the current accuracy and fallback rate baseline for the bots in production?"** Asking for baselines signals that you think in measurable outcomes. Most candidates ask about tech stack instead; very few ask where the numbers currently are.
- **"How do you catch a quality regression before a user does?"** Steers the conversation toward evaluation, which is your strongest area and a genuine concern for any enterprise AI team.
- **"How is the AI roadmap structured — central platform team, or does each business unit own its own bots?"** Shows you are thinking at the organisational level, not just the component level.
- **"How much of the assistant build is authored in Co-Pilot Studio today, versus code-first?"** Establishes where you'd actually add value, and is a smart way to surface the ramp question without being asked.
- **"What's the biggest technical challenge the team is wrestling with right now?"** A senior panel respects someone who wants to hear the real problems. Often leads to the most useful conversation of the interview.
- **"How do you handle a policy document change that invalidates a lot of embeddings at once?"** Highly specific, and it directly demonstrates you have operated this. Very few candidates can ask it convincingly.

THE STRATEGIC VERSION OF THIS

Ask the question that matches the weakness you want to be seen overcoming. If you lead with the evaluation question, the panel spends more of the interview discussing the area where you are strongest — which is not manipulation, it is simply making sure the interview covers your real strengths.

Pick at most three. Firing eight questions reads as nervous, or as though you already know the job and are interviewing them.